LUMBINI TECHNOLOGICAL UNIVERSITY

Institute of Engineering and Information Technology

Nepalgunj, Banke

A PROJECT REPORT ON

Mero-Uni (Note Sharing Platform)

Submitted to:

Department of B.Tech in Computer Science and Artificial Intelligence

In partial fulfillment of the requirement for the B.Tech in Computer Science and Artificial Intelligence

Submitted by:

Adarsh Sharma

Ritika Khadka

Manas Rana Magar

Under the Supervision of:

Pratik Chand

Senior Assistant Professor, LTU

LUMBINI TECHNOLOGICAL UNIVERSITY

Institute of Engineering and Information Technology

Supervisor Recommendation

I, hereby recommend that this project is completed under my supervision by **Adarsh Sharma (23102001), Ritika Khadka (23102010), Manas Rana Magar (23102005)** entitled "**Mero-uni (Note Sharing Platform)"** in partial fulfillment of the requirements for the degree of B. Tech in Computer Science and Artificial Intelligence is recommended for the final evaluation.

……..……

Signature

Pratik Chand

Senior Assistant Professor, LTU

B.Tech in Computer Science &
Artificial Intelligence

Nepalgunj, Banke

LUMBINI TECHNOLOGICAL UNIVERSITY

Institute of Engineering and Information Technology



LETTER OF APPROVAL

This is to certify that this project is completed by **Adarsh Sharma (23102001), Ritika Khadka (23102010), Manas Rana Magar (23102005)** entitled "**Mero-Uni Note Sharing Platform"** in partial fulfillment of the requirements for the degree of B. Tech in Computer Science and Artificial Intelligence has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

……
Signature of Supervisor
Pratik Chand
Senior Asst. Professor
B.Tech in CS & AI
Nepalgunj, Banke

……
Signature of Program Coordinator
Pratik Chand
Senior Asst. Professor
B.Tech in CS & AI
Nepalgunj, Banke

……
Signature of Internal Examiner

……
Signature of External Supervisor

ABSTRACT

The Mero-Uni project addresses the critical challenge of fragmented and inefficient academic resource sharing among students at Lumbini Technological University (LTU). Current methods, relying on informal channels like messaging apps and physical transfers, result in disorganized, inconsistent, and often outdated availability of essential study materials. This project introduces Mero-Uni, a web-based academic resource-sharing platform specifically designed for the LTU community, particularly students of Computer Science and Artificial Intelligence (CS&AI). Developed using the Django framework with Python and a MySQL database, Mero-Uni provides a centralized, secure, and intuitively user-friendly environment. Key functionalities include administrators to upload and moderate content, and advanced search and categorization features (by semester, subject, and resource type). The platform ensures the quality and accessibility of academic resources, such as lecture notes, past examination papers, and lab manuals. The system analysis and design phase utilized the Object-Oriented approach, incorporating a Use-Case Diagram to define system boundaries, Class, Object, State and a Sequence Diagram to illustrate key user interactions. The implementation phase focused on developing core modules for resource management, followed by rigorous Unit and System Testing to ensure reliability and functional correctness. By centralizing resource distribution and fostering a collaborative learning environment, Mero-uni significantly enhances academic efficiency and supports the effective transfer of institutional knowledge across student cohorts at LTU.

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to Lumbini Technological University for providing us an opportunity to do our project work on “Mero-Uni”. This project bears on the thoughts and opinions of many people. We sincerely thank our senior assistant professor ‘Mr. Pratik Chand’ for guidance and encouragement in carrying out this project work.

We also sincerely thank Lumbini Technological University (LTU) for providing an opportunity to enhance our skills and knowledge by providing practical ways of study through this project program and for constantly monitoring our work and setting up precise deadlines.

Our sincere thanks also go to our Supervisor Mr. Pratik Chand, for their support and cooperation. I express my thanks to my friends for their advice and support, without whom it would have been difficult for completion of this project.

Sincerely,

Adarsh Sharma (23102001)

Ritika Khadka (23102010)

Manas Rana Magar (23102005)

Table of Contents

ABSTRACT iv
ACKNOWLEDGEMENT v
List of Figures vii
List of Tables viii
List of Abbreviation ix
Chapter 1: Introduction 1
1.1. Introduction 1
1.2. Problem Statement 2
1.3. Objectives 2
1.4. Scope and Limitation 2
1.4.1 Scope 2
Chapter 2: Background Study and Literature Review 5
2.1. Background Study 5
2.2. Literature Review 6
Chapter 3: System Analysis and Design 8
3.1. System Analysis 8
3.1.1. Requirement Analysis 8
3.1.2. Feasibility Analysis 12
3.1.3. System Modeling 16
3.2 System Design 23
3.2.1 Refinement of class and Object Diagram 23
3.2.2 Component Diagram 25
3.2.3 Deployment Diagram 26
Chapter 4: Implementation and Testing 27
4.1. Implementation 27
4.1.2. Implementation Details of Modules 28
4.2. Testing 30
4.2.4. Testing Tools 34
Chapter 5: Conclusion and Future Recommendations 35
5.1. Lesson Learned and Outcome 35
5.2. Conclusion 36
5.3. Future Recommendations 37
References 38
Appendix 39

List of Figures

Figure 1 Use Case Diagram 8
Figure 2 Gantt Chart 14
Figure 3 Class Diagram 16
Figure 4 Object Diagram 18
Figure 5 State Diagram 19
Figure 6 Sequence Diagram 20
Figure 7 Activity Diagram 22
Figure 8 Refined Class Diagram 23
Figure 9 Refined Object Diagram 24
Figure 10 Component Diagram 25
Figure 11 Deployement Diagram 1 26
Figure 12Deployment Diagram 2 26
Figure 13 Homepage 39
Figure 14 Semester Page 40
Figure 15 Subject Page 40
Figure 16 Resource Page 41
Figure 17 About Page 41
Figure 18 Database Mero-uni 42
Figure 19 Log Records 1 52
Figure 20 Log Record 2 53

List of Tables

Table 1 Use-Case Descriptions 9
Table 2 Testing 30
Table 3 Unit Testing 34

List of Abbreviation

Abbreviation	Full Form
AI	Artificial Intelligence
B. Tech	Bachelor of Technology
CSAI	Computer Science and Artificial Intelligence
IT	Information Technology
LMS	Learning Management System
LTU	Lumbini Technological University
TAM	Technology Acceptance Model
TU	Tribhuvan University
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
SQL	Structured Query Language
ORM	Object Relational Mapping
MVT	Model View Templates
DB	Database

Chapter 1: Introduction

1.1. Introduction

In the academic landscape, efficient access to and management of educational resources are the most necessities for student success. However, students frequently encounter significant challenges in accessing, organizing, and sharing essential academic resource materials such as lecture notes, past examination papers, laboratory reports, and study guides. Current methods, including messaging apps, emails, and physical handouts, are fragmented, inefficient, and often unreliable. This disorganization hinders students' ability to access up-to-date materials, particularly during high-pressure exam time, leading to increased stress, incomplete preparation, and reduced academic performance. Furthermore, these systemic inefficiencies hinder effective peer collaboration, limiting the potential for collective learning within academic communities.

To address these challenges, this report introduces **Mero-uni**, a web-based academic resource-sharing platform specifically tailored for the CS&AI student of Lumbini Technological University (LTU). Mero-uni provides a centralized, secure, and intuitively user-friendly environment where administrators can upload, organize, and distribute study materials. Key features, including structured categorization by semester and subject, advanced search capabilities, and robust access controls, are integrated to streamline resource management and ensure equitable access to high quality educational resource materials.

By fostering enhanced collaboration and significantly reducing barriers to resource accessibility, Mero-Uni aims to elevate academic achievement, enrich the overall learning experience, and strengthen the academic fabric of the LTU community.

1.2. Problem Statement

The current ecosystem for academic resource sharing at Lumbini Technological University is characterized by several critical deficiencies that collectively undermine student learning and administrative efficiency. Firstly, fragmented resource sharing is relying on informal channels such as WhatsApp groups, Google Drive, and physical pen drives. This leads to a highly disorganized and inconsistent availability of essential academic materials, making it ambiguous for students to access the most current and accurate versions of materials.

Secondly, there is a profound lack of standardization in the repository of LTU-specific notes, past papers, and lab manuals. The absence of a centralized and curated system results in wide variations in the accuracy and quality of shared content, making content verification a significant challenge for users. This lack of quality control can lead to reliance on outdated or incorrect information, negatively impacting academic outcomes.

Lastly, a notable issue is exam-time scarcity, where junior students frequently struggle to obtain relevant and timely notes from their seniors. The absence of a structured and efficient mechanism for accessing exam focused content increases stress during critical study periods and limits the effective transfer of institutional knowledge across student. These problems collectively highlight an urgent need for a unified, reliable, and accessible platform to manage and disseminate academic resources within the LTU community.

1.3. Objectives

i. To provide a centralized web-based platform for sharing academic notes and resources among LTU students.
ii. To streamline the process of uploading, accessing, and managing study materials by semester and subject.

1.4. Scope and Limitation

1.4.1 Scope

The Mero-uni project encompasses the development of a web-based application designed to streamline academic resource sharing exclusively for the Lumbini Technological University community. The platform will enable registered users (administrators) to upload, download, and manage various academic resources. Key functionalities within the scope include:

Resource Upload and Management: Administrators can upload files (e.g., PDF) and categorize them by semester, subject, and resource type (e.g., notes, question banks, lab solutions, assignments, past papers, syllabus).

Resource Browsing and Search: Students can browse resources by semester, subject, and resource type, and utilize a comprehensive search function to find specific materials based on keywords in titles, descriptions, or associated metadata.

Resource Download: users can download available resources, with download counts tracked for each resource.

The system is developed using Python with the Django framework for the backend and a relational database (e.g., MySQL) for database. The frontend will utilize standard web technologies (HTML, CSS, JavaScript) to provide a responsive user interface. The primary focus is on creating a functional and stable platform for resource dissemination within the LTU academic context.

1.4.2 Limitation

Mero-uni significant challenges in academic resource sharing, the current iteration of the project is subject to certain limitations, primarily due to project scope and resource constraints:

i. The system lacks real-time collaboration and editing features for shared documents.
ii. It is web-only, without offline access or native mobile applications.
iii. The current version lacks advanced features like notifications, version control, and user feedback mechanisms.

1.5. Report Organization

This report is systematically organized into five chapters to provide a comprehensive and coherent understanding of the Mero-uni project. Chapter 1, the Introduction, establishes the context of the project by outlining the problem statement, defining the project's objectives, and expressing its scope and limitations. Chapter 2, Background Study and Literature Review, provides foundational knowledge regarding the technologies employed and critically analyzes existing research and solutions to academic resource sharing. Chapter 3, System Analysis and Design, details the functional and non-functional requirements, presents a thorough feasibility analysis, and illustrates the system's architecture through various modeling diagrams. Chapter 4, Implementation and Testing, describes the practical aspects of the project, including the tools utilized and the methodologies for testing the developed system. Finally, Chapter 5, Conclusion and Future Recommendations, summarizes the project's outcomes, discusses lessons learned, and proposes potential avenues for future development and research. This structured approach ensures a logical progression of information, facilitating a clear and in-depth understanding of the Mero-uni platform.

Chapter 2: Background Study and Literature Review

2.1. Background Study

The development of the Mero-uni academic resource-sharing platform is supported by a robust selection of technologies and a foundational understanding of web application architecture. At its core, Mero-uni leverages the Django framework, a high level Python web framework renowned for its rapid development capabilities, clean design, and pragmatic approach to web application construction [1]. Django adheres to the Model-View-Template (MVT) architectural pattern, which effectively separates the application's data logic (Model), presentation logic (Template), and business logic (View). This separation of concerns significantly enhances code maintainability, scalability, and reusability, making it an ideal choice for complex web projects like Mero-uni.

Key features of Django that are particularly advantageous for Mero-uni include its powerful Object-Relational Mapper (ORM), which abstracts database interactions, allowing developers to work with Python objects instead of raw SQL queries. This not only accelerates development but also improves database portability and reduces the likelihood of SQL injection vulnerabilities. Furthermore, Django's built-in administrative interface provides a ready-to-use backend for content management, which is crucial for administrators to upload, categorize, and moderate resources efficiently. Its robust security features, including protection against cross-site scripting (XSS), cross-site request forgery (CSRF), and SQL injection, are vital for safeguarding sensitive academic data and user information.

Complementing Django, SQL (Structured Query Language) serves as the backbone for data persistence within Mero-uni. While Django's ORM handles the direct interaction, SQL remains the fundamental language for managing and querying relational databases. In the context of Mero-uni, SQL databases store critical information such as semester details, subject classifications, resource metadata, and file locations. The choice of a relational database ensures data integrity, consistency, and the ability to perform complex queries necessary for efficient resource retrieval and categorization. Django's ORM translates Python object manipulations into optimized SQL queries, ensuring seamless data storage and retrieval without requiring manual SQL scripting for most operations.

Beyond the specific technologies, the conceptual framework of Mero-uni is rooted in the broader domain of academic resource sharing platforms. These platforms have evolved significantly with the advent of digital technologies, transitioning from traditional physical libraries and informal peer-to-peer exchanges to sophisticated online repositories. The importance of such platforms in modern education cannot be overstated. They serve as central hubs for knowledge dissemination, facilitating equitable access to learning materials, promoting collaborative learning environments, and supporting self-directed study. The challenges these platforms aim to solve are universal: overcoming geographical barriers, ensuring timely access to information, standardizing content organization, and providing reliable storage for educational assets. Mero-uni seeks to contribute to this evolving landscape by offering a tailored solution that addresses the specific needs and challenges of the LTU academic community

2.2. Literature Review

The rapid increase of digital technologies has profoundly transformed educational paradigms, making online academic resource sharing platforms indispensable tools for modern learning environments [2]. This literature review examines existing research and systems related to digital resource management and collaborative learning, providing a contextual backdrop for the Mero-uni project and highlighting the gaps it aims to address.

Early forms of digital resource sharing often relied on rudimentary file-sharing mechanisms or institutional learning management systems (LMS) that, while centralizing content, frequently lacked user-friendly interfaces or robust search capabilities [3]. For instance, studies by Smith and Jones (2018) [4] highlighted that while LMS platforms provided a repository, their rigid structures often hindered informal peer-to-peer sharing, leading students to resort to less secure and unorganized methods like social media groups or cloud storage services. This fragmentation, as observed in the current LTU context, results in inconsistent resource availability and significant challenges in verifying content accuracy.

More recent research has focused on the development of dedicated academic resource platforms that emphasize user experience, content organization, and collaborative features.

work by Chen et al. (2020) [5] on a university-specific knowledge-sharing platform demonstrated the importance of intuitive categorization and powerful search functionalities in enhancing resource discoverability. Their findings suggest that a well-designed

information architecture significantly reduces the time students spend searching for materials, thereby improving study efficiency.

Furthermore, the concept of user-generated content and peer-to-peer learning has gained traction. Research by Lee and Kim (2019) [6] explored the dynamics of student-contributed resource platforms, noting that while such systems foster a sense of community and ownership, they also necessitate robust moderation mechanisms to ensure content quality and accuracy. This highlights a critical design consideration for Mero-uni: balancing the benefits of user contributions with the need for reliable, verified information.

Another relevant area of research involves the impact of technology on academic performance. A study by Johnson and Miller (2021) [7] indicated a positive correlation between access to organized digital learning resources and improved student grades, particularly in courses requiring extensive material review. This reinforces the academic imperative behind Mero-uni, positioning it as a tool to directly enhance student outcomes by mitigating the disadvantages of fragmented resource access.

Finally, the literature on information systems security and data privacy is highly relevant. Given that Mero-uni will handle educational content and user data, adherence to best practices in web security is paramount [8]. Studies by Davis and White (2020) underscore the importance of secure authentication, data encryption, and regular security audits in maintaining user trust and protecting sensitive information in online platforms. Mero-uni's reliance on Django's built-in security features aligns with these recommendations. [9]

In summary, while existing literature highlights various aspects of digital resource management and collaborative learning, there remains a need for tailored, secure, and user-friendly platforms that specifically address the unique challenges of academic resource sharing within a university context. Mero-uni aims to synthesize the best practices identified in the literature, providing a centralized and efficient solution for the LTU community.

Chapter 3: System Analysis and Design

3.1. System Analysis

System analysis is a crucial phase in the software development lifecycle, focusing on understanding the needs of the users and the functionalities required from the system. This involves a detailed examination of the problem domain, identification of requirements, and assessment of feasibility. For the Mero-uni project, this phase laid the groundwork for designing a robust and user-centric academic resource-sharing platform.

3.1.1. Requirement Analysis

Requirement analysis involves identifying, documenting, and validating the needs of the stakeholders. For Mero-uni, requirements were categorized into functional and non-functional requirements, providing a comprehensive understanding of what the system must do and how well it must perform.

I. Functional Requirement

Functional requirements define the specific actions or behaviors the system must perform. These are directly tied to the core functionalities that Mero-uni will offer to its users. The primary functional requirements identified for the Mero-uni platform are:

Use-Case Diagram

The use case diagram provides a high-level view of the Mero-uni system's functional requirements by illustrating the interactions between external actors and the systems use cases. It helps in understanding what the system does from the perspective of its users

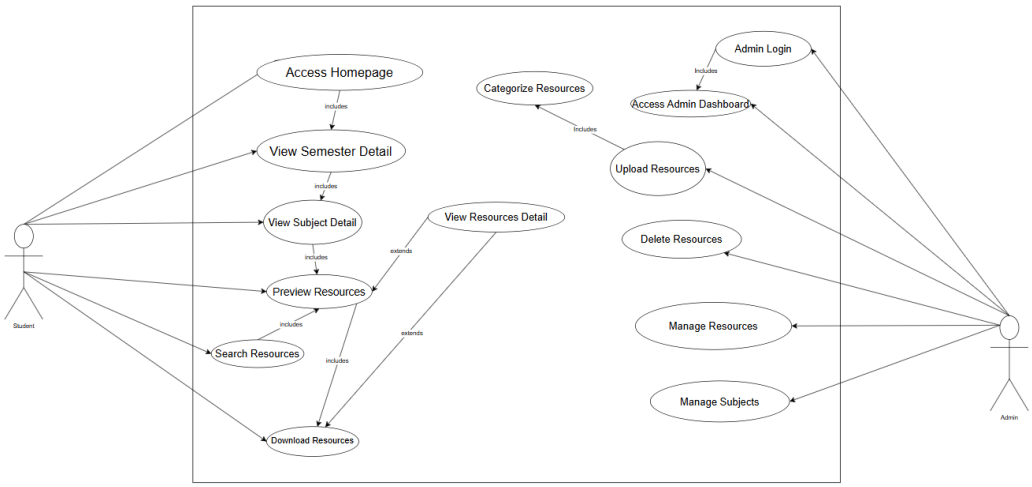


Figure 1 Use Case Diagram

Use-case Description

Table 1 Use-Case Descriptions

Use Case Name	Actor	Description	Preconditions	Post conditions
Upload Resources	Admin	Admin uploads resources to the system.	Admin is logged in.	Resources are successfully uploaded and available.
Categorize Resources	Admin	Admin categorizes the uploaded resources.	Resources are uploaded.	Resources are properly categorized.
Delete Resources	Admin	Admin removes resources from the system.	Resources exist in the system.	Resources are deleted and no longer accessible.
Manage Resources	Admin	Admin manages every resources	Resources may or may not exists	Resources are managed, manipulated or CRUD
Manage Semester	Admin	Admin manages semester-related data.	Admin is logged in.	Semester data is updated or created.
Manage Subject	Admin	Admin manages subject-related data.	Admin is logged in.	Subject data is updated or created.
View Subjects	User	User views the list of subjects.	User is browser.	Subject list is displayed to the user.
View Semester	User	User views semester-related information.	User is browser.	Semester information is displayed.
View Resources	User	User views available resources.	Resources are available.	Resources are displayed to the user.
Search Resources	User	User searches for specific resources.	Resources are available.	Search results are displayed.
Preview Resources	User	User previews resources before download.	Resources are available.	Resource preview is displayed.
Download Resources	User	User downloads selected resources.	Resources are available.	Resources are downloaded successfully.

User Authentication and Authorization: The system does not require users (except admin). only administrators should have the privilege to upload and categorize resources.

Resource Upload and Management: Administrators must be able to upload various types of academic resources (e.g., lecture notes, past papers, lab solutions) to the platform. This includes specifying metadata such as title, description, associated subject, semester, and resource type. The system must support common file formats (e.g., PDF) and provide mechanisms for administrators to edit or delete uploaded resources.

Resource Categorization and Organization: Resources must be systematically categorized by academic semester and subject. The system should allow for the creation and management of resource types (e.g., 'Notes', 'Question Bank', 'Syllabus') to facilitate structured organization and easy retrieval. This hierarchical categorization ensures that users can navigate through materials logically.

Resource Browsing and Navigation: Users must be able to browse resources intuitively. This includes navigating through different semesters to view associated subjects, and then viewing resources within a specific subject. The system should provide clear pathways for exploration, such as dedicated pages for each semester, subject, and resource type.

Advanced Search Functionality: The system must provide a robust search engine that allows users to find resources based on keywords. The search should be capable of querying resource titles, descriptions, subject names, subject codes, and resource types. Search results should be presented clearly, allowing users to quickly identify relevant materials.

Resource Detail Viewing: Upon selecting a resource, users must be able to view detailed information about it, including its title, description, associated subject, resource type, and the user who uploaded it. This page should also provide the option to download the resource.

Resource Download: Students must be able to download selected resources. The system must accurately track the number of downloads for each resource, providing valuable metrics on resource utility. The download process should be secure and reliable, ensuring file integrity.

II. Non-Functional Requirements

Non-functional requirements specify criteria that can be used to judge the operation of a system, rather than specific behaviors. They are crucial for ensuring the quality, usability, and maintainability of the Mero-uni platform.

Performance:

Response Time: The system should load pages within 2-3 seconds under normal load conditions (up to 50 concurrent users).

Search Latency: Search queries should return results within 1-2 seconds.

Download Speed: Resource downloads should initiate promptly, with speeds limited by network bandwidth rather than server processing.

Security:

Authorization: Access to administrative functions (e.g., resource upload, deletion) must be strictly restricted to authorized administrator accounts.

Data Integrity: Mechanisms must be in place to prevent unauthorized modification or corruption of resource files and metadata.

Vulnerability Protection: The system must be resilient against common web vulnerabilities such as SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

Usability:

Intuitive Interface: The user interface must be clean, intuitive, and easy to navigate for both students and administrators.

Accessibility: The platform should adhere to basic web accessibility guidelines to ensure usability for a diverse range of users.

Error Handling: The system should provide clear, user-friendly error messages and guidance for recovery.

Reliability:

Uptime: The system should maintain an uptime of at least 99.5% during operational hours.
Data Backup: Regular backups of the database and uploaded files must be performed to prevent data loss.

Consistency: Data displayed across different parts of the application (e.g., resource details, search results) must be consistent and up-to-date.

Scalability:

User Load: The system should be designed to accommodate a growing number of users (e.g., up to 5000 users) and increasing concurrent access without significant performance degradation.

Resource Volume: The architecture should support a large volume of uploaded resources (e.g., thousands of files) and efficient retrieval.

Maintainability:

Code Quality: The codebase must be well-structured, modular, and adhere to established coding standards to facilitate future enhancements and bug fixes.

Documentation: Comprehensive technical documentation for the codebase and system architecture should be maintained.

Ease of Deployment: The system should be straightforward to deploy and configure in a production environment. Portability:

Browser Compatibility: The web interface should be compatible with major modern web browsers (e.g., Chrome, Firefox, Edge, Safari).

3.1.2. Feasibility Analysis

Feasibility analysis assesses the practicality and viability of the proposed system. For the Mero-uni project, a comprehensive analysis was conducted across technical, operational, economic, and schedule dimensions to ensure that the project is achievable within the given constraints and resources.

I. Technical Feasibility

Technical feasibility evaluates whether the proposed system can be developed and implemented using existing technology and within the technical capabilities of the development team. The Mero-uni project is technically feasible due to several factors:

Proven Technologies: The project relies on well-established and mature technologies: Python, Django, and SQL databases. These technologies have extensive documentation, large community support, and a wealth of readily available libraries and tools, minimizing technical risks.

Team Expertise: The development team possesses the necessary skills in Python programming, Django framework development, web development (HTML, CSS, JavaScript), and database management. Any minor skill gaps can be addressed through readily available online resources and collaborative learning.

Infrastructure Availability: The required development and deployment infrastructure (e.g., development environments, potential cloud hosting for deployment) are accessible and cost-effective. Integration Simplicity: Django's ORM simplifies database integration,

and its modular structure facilitates the integration of various components without significant compatibility issues.

II. Operational Feasibility

Operational feasibility assesses how well the proposed system will solve the current problems and how it will be integrated into the existing organizational environment. Mero-uni demonstrates strong operational feasibility:

Problem Resolution: The platform directly addresses the identified problems of fragmented resource sharing, lack of standardization, and exam-time scarcity by providing a centralized, organized, and accessible repository for academic materials.

User Acceptance: The intuitive design and user-friendly interface are expected to promote high user adoption among LTU students and administrators. The system is designed to simplify existing workflows for resource management and access.

Minimal Disruption: The web-based nature of the application means minimal disruption to existing IT infrastructure. Users can access the platform via standard web browsers without requiring specialized software installations.

Training Requirements: Training requirements for end-users (students) are minimal due to the intuitive design. Administrators will require basic training on the content management system, which can be easily provided.

III. Economic Feasibility

Economic feasibility evaluates the financial viability of the project, weighing the costs against the benefits. For an academic project, the economic analysis focuses on resource allocation and potential long-term value.

Development Costs: Development costs are primarily associated with human resources (developer time). The use of open-source technologies (Python, Django) significantly reduces software licensing costs. Development tools and environments are also largely open-source or freely available.

Operational Costs: Ongoing operational costs would include server hosting (if deployed), domain registration, and potential maintenance. These costs are relatively low for a web application of this scale.

Tangible Benefits: While direct revenue generation is not an objective, Mero-uni offers significant tangible benefits, including reduced time spent by students searching for resources, improved academic performance leading to higher student retention, and enhanced administrative efficiency in managing educational content.

Intangible Benefits: Intangible benefits include improved student satisfaction, enhanced reputation for LTU as a technologically progressive institution, fostering a stronger academic community, and better knowledge transfer between student cohorts.

Given the low development and operational costs coupled with significant tangible and intangible benefits, the Mero-uni project is economically feasible.

IV. Schedule Feasibility

Schedule feasibility assesses whether the project can be completed within the allocated timeframe. The Mero-uni project is designed to be completed within the academic semester, adhering to the following timeline considerations:

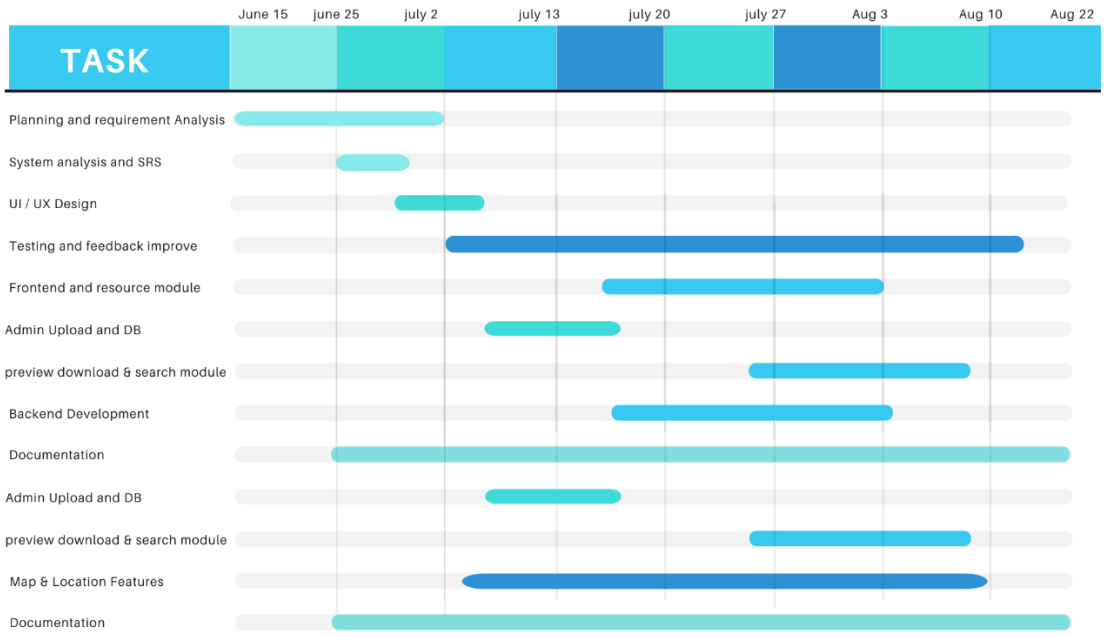


Figure 2 Gantt Chart

Defined Milestones: The project plan includes clear milestones for proposal submission, mid-term progress evaluation, and final submission, as outlined in the BAI290 Project I Syllabus. These milestones provide a structured approach to project management.

Agile Approach: While not strictly agile, the development process incorporates iterative cycles of design, implementation, and testing, allowing for flexibility and adaptation within the given schedule.

Resource Allocation: The project team's dedicated effort and efficient task allocation are critical for meeting deadlines. The modular nature of the Django framework also allows for parallel development of different components.

Contingency Planning: Buffer time is allocated within the overall schedule to account for unforeseen challenges or technical complexities, ensuring that minor setbacks do not affect the entire project timeline.

Based on the structured approach, available resources, and the nature of the project, Mero-uni is deemed feasible within the academic schedule.

3.1.3. System Modeling

System modeling is a critical aspect of system analysis and design, providing visual representations of the system's structure, behavior, and interactions. For the Mero-uni project, various UML (Unified Modeling Language) diagrams were utilized to illustrate different facets of the system. These diagrams are essential for translating abstract requirements into concrete design specifications

3.1.3.1 Class and Object Diagram*:*

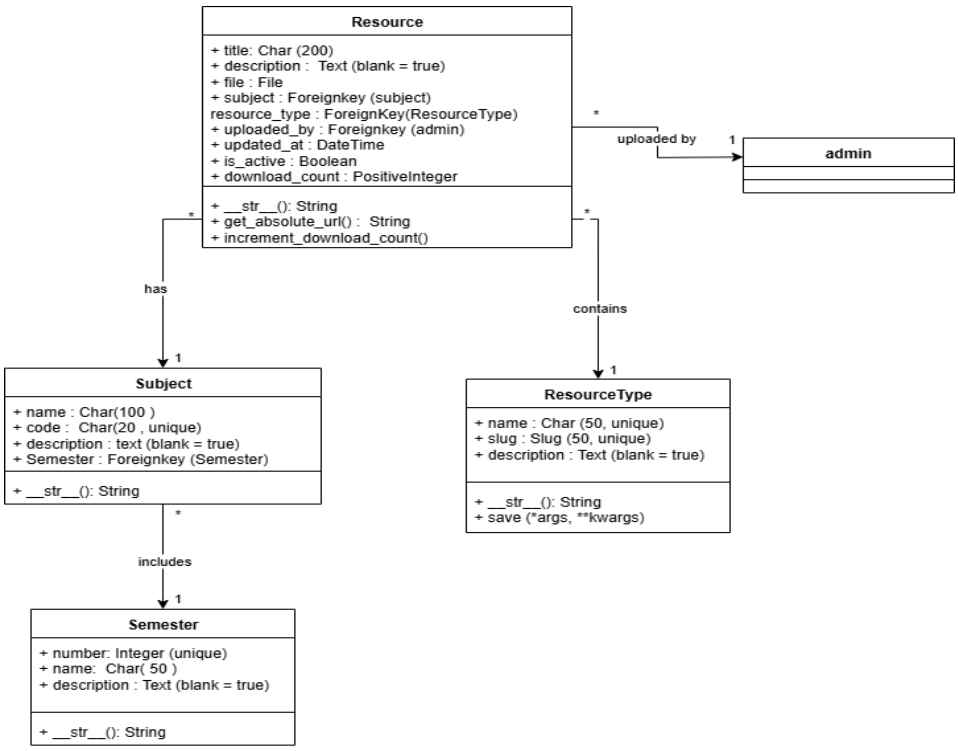


Figure 3 Class Diagram

The class diagram provides a static, structural view of the Mero-uni project's data models, as defined in models.py. It illustrates the classes, their attributes, and the relationships between them, forming the blueprint of the application's data architecture. The provided UML class diagram represents a system for managing resources, It includes four main classes: Resource, Subject, Semester, and Resource Type, along with their relationships and attributes.

Classes and Attributes

a) Resource

I. Attributes:

i. title: Char (200): The title of the resource.

ii. description: Text (blank = true): A description of the resource (can be blank).

iii. subject: ForeignKey(subject): A foreign key linking to the Subject class.

iv. resourceType: ForeignKey(ResourceType): A foreign key linking to the ResourceType class.

v. uploaded_by: ForeignKey(admin): A foreign key indicating the admin who uploaded the resource.

vi. upload_dt: DateTime: The date and time when the resource was uploaded.

vii. upload_count: PositiveInteger: The number of times the resource has been uploaded.

viii. get_absolute_url(): A method to get the absolute URL of the resource.

ix. increment_download_count(): A method to increment the download count.

b) Subject

I. Attributes:

i. name: Char(100) (unique): The unique name of the subject.

ii. description: Text (blank = true): A description of the subject (can be blank).

iii. semester: ForeignKey(Semester): A foreign key linking to the Semester class.

iv. slug: Slug: A URL-friendly version of the name.

c) Semester

I. Attributes:

i. name: Char(100) (unique): The unique name of the semester.

ii. description: Text (blank = true): A description of the semester (can be blank).

iii. slug: Slug: A URL-friendly version of the name.

d) Resource Type

I. Attributes:

i. name: Char (50) (unique): The unique name of the resource type.

ii. slug: Slug (unique = true): A unique URL-friendly version of the name.

iii. description: Text (blank = true): A description of the resource type (can be blank).

iv. save(*args, **kwargs): A method to save the resource type with additional arguments.

Relationships

a. **Unidirectional Association:** The arrow from Resource to admin indicates that a Resource is uploaded by an admin, but the relationship is not navigable in the reverse direction.

b. **"has" Relationship:** Resource has a Subject, suggesting that each resource is associated with one subject.

c. **"contains" Relationship:** Resource contains a Resource Type, indicating that each resource is of a specific type.

d. **"includes" Relationship:** Semester includes Subject, meaning a semester can contain multiple subjects.

Interpretation

This diagram models a system where:

a. An admin uploads Resources, which are categorized by Subject and Resource Type.

b. Each Resource has metadata like title, description, and upload details, and tracks download counts.

c. Subjects are organized under Semesters, with each having a unique name and optional description.

d. Resource Type defines different types of resources, also with unique names and descriptions.

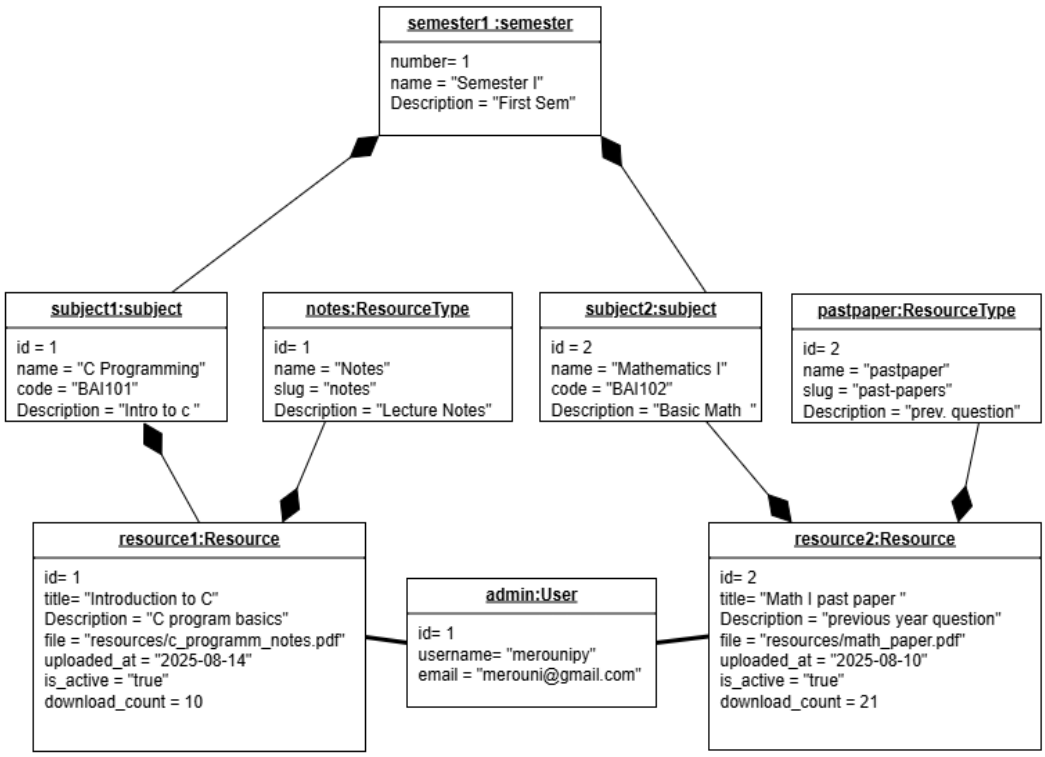


Figure 4 Object Diagram

3.1.3.2 State and Sequence Diagram

The state diagram illustrates the dynamic behavior of the Merouni application from a user's perspective, showing the different states a user can be in and the transitions between these states based on user actions. It provides a high-level overview of the user journey through the application's main functionalities.

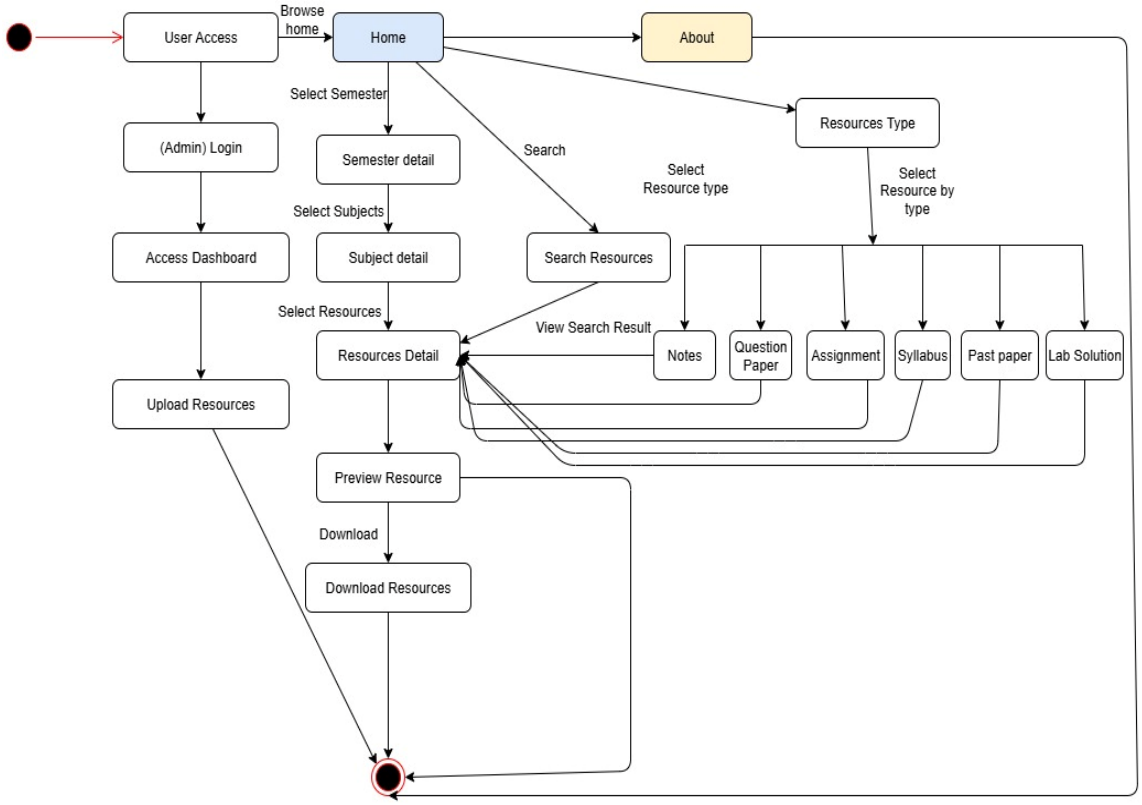


Figure 5 State Diagram

Explanation:

Home: This is the initial state upon entering the application, displaying an overview of semesters, recent resources, and resource types.

Semester Detail: Accessed by selecting a specific semester from the Home page, showing subjects within that semester.

Subject Detail: Accessed by selecting a subject from the Semester Detail page, displaying resources related to that subject.

Resource Detail: Accessed by selecting a resource from the Subject Detail page or search results, showing detailed information about the resource.

Download Resource: A transient state initiated from the Resource Detail page, leading to the download of the resource file.

Search Resources: A state entered when the user initiates a search query, displaying search results.

ResourcesByType: A state entered when the user filters resources by a specific type, displaying resources of that type.

About: A state representing the viewing of the about page.

Transitions: Arrows indicate transitions between states triggered by user actions (e.g., "Select Semester," "Search," "Download File").

Sequence Diagram

The sequence diagram illustrates the interactions between different components of the Mero-uni system over time, focusing on the order of messages exchanged to accomplish specific functionalities. It provides a dynamic view of how objects collaborate to perform a task.

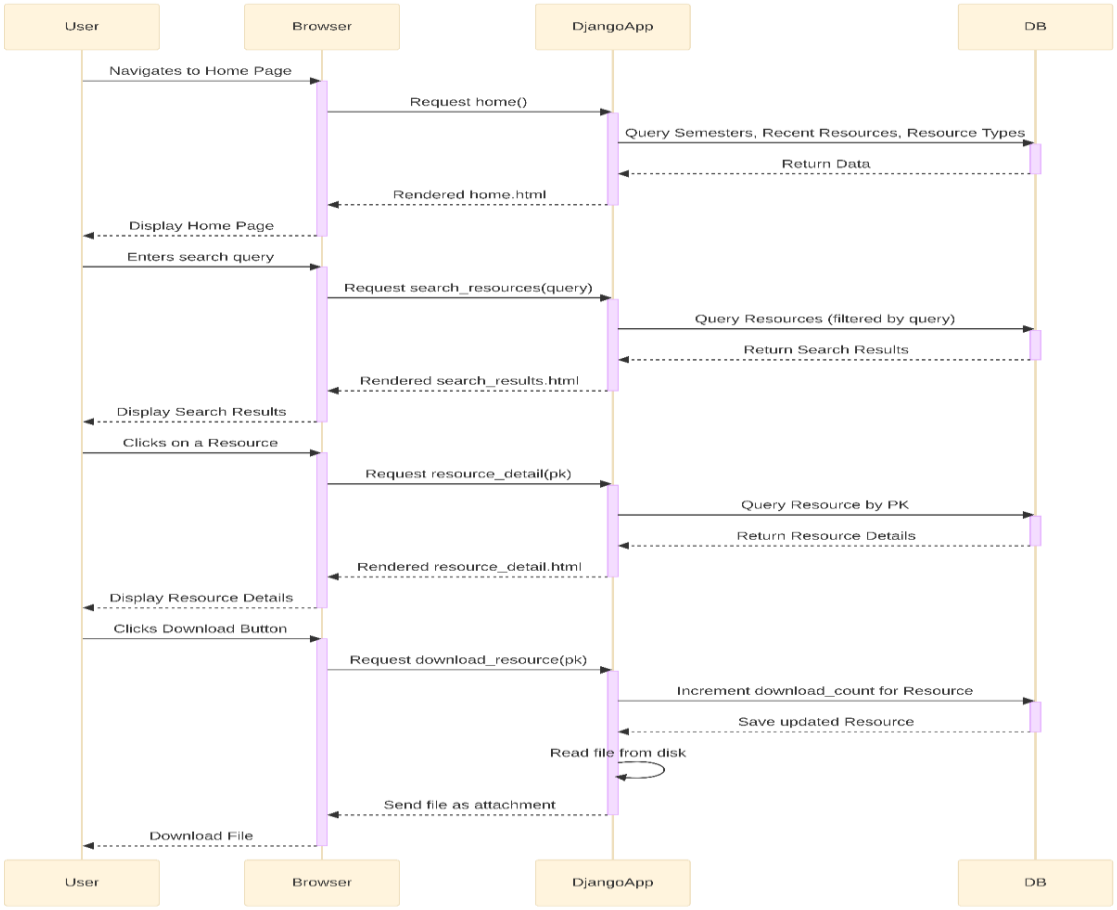


Figure 6 Sequence Diagram

Explanation:

Participants: The diagram defines four key participants: User (the end-user), Browser (the web browser client), DjangoApp (the Mero-uni Django application, specifically its views and business logic), and DB (the database where Mero-uni data is stored).

Message Flow: Arrows represent messages or function calls between participants. Solid lines with arrowheads indicate synchronous calls, while dashed lines with arrowheads indicate return messages.

Home Page Navigation: When a user navigates to the home page, the browser sends a request to the Django application. The Django application then queries the database for necessary data (semesters, recent resources, resource types), receives the data, renders the home.html template, and sends the rendered page back to the browser for display to the user.

Search Functionality: When a user enters a search query, the browser sends the request to the Django application. The application queries the database with the filter, receives the results, renders the search_results.html template, and displays it to the user.

Resource Detail View: Clicking on a resource sends a request to the Django application, which queries the database for the specific resource details. The details are returned, the resource_detail.html template is rendered, and the page is displayed to the user.

Resource Download: When a user clicks the download button, the Django application first increments the download count in the database. After saving the updated resource, it reads the file from the server's disk and sends it as an attachment to the browser, initiating the download for the user.

3.1.3.3 Activity Diagram

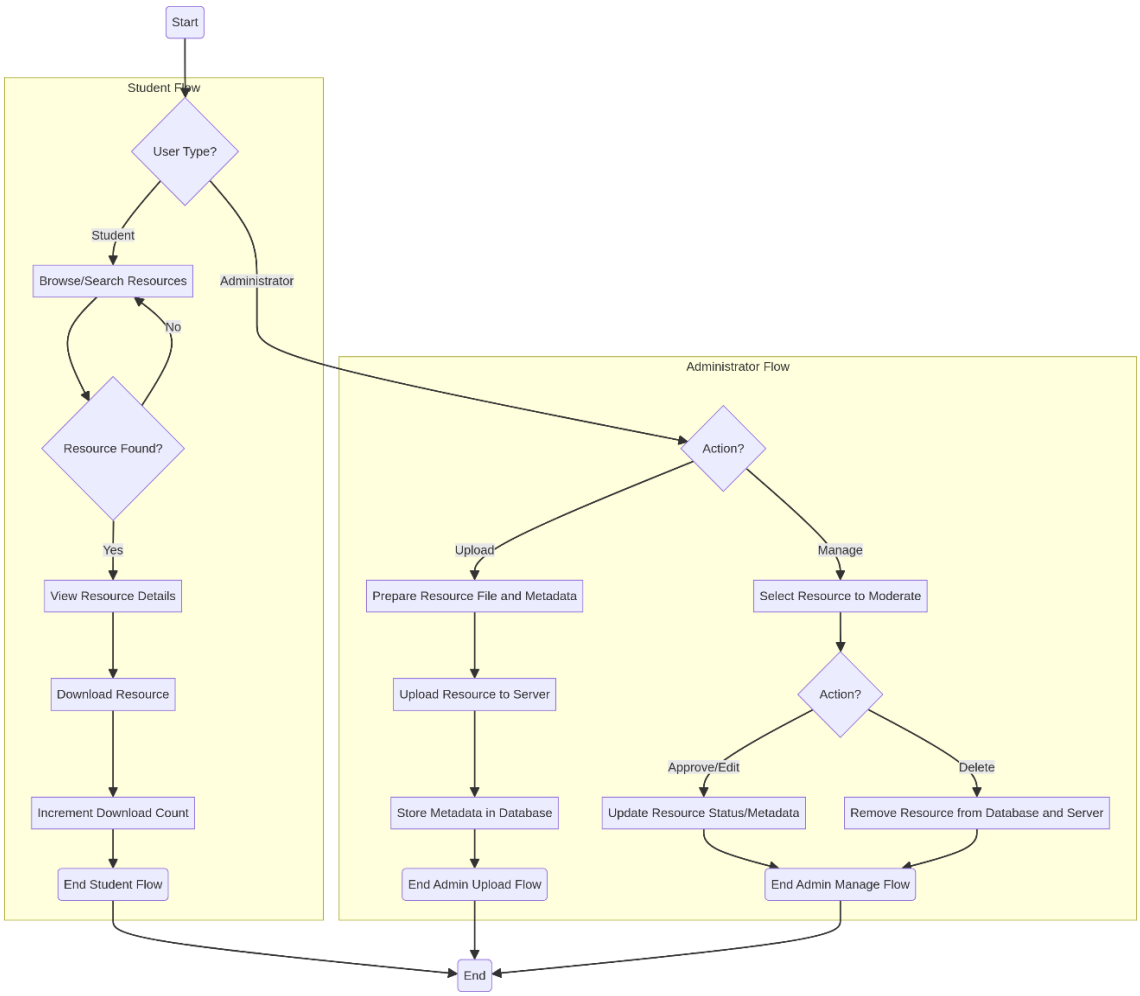


Figure 7 Activity Diagram

3.2 System Design

3.2.1 Refinement of class and Object Diagram

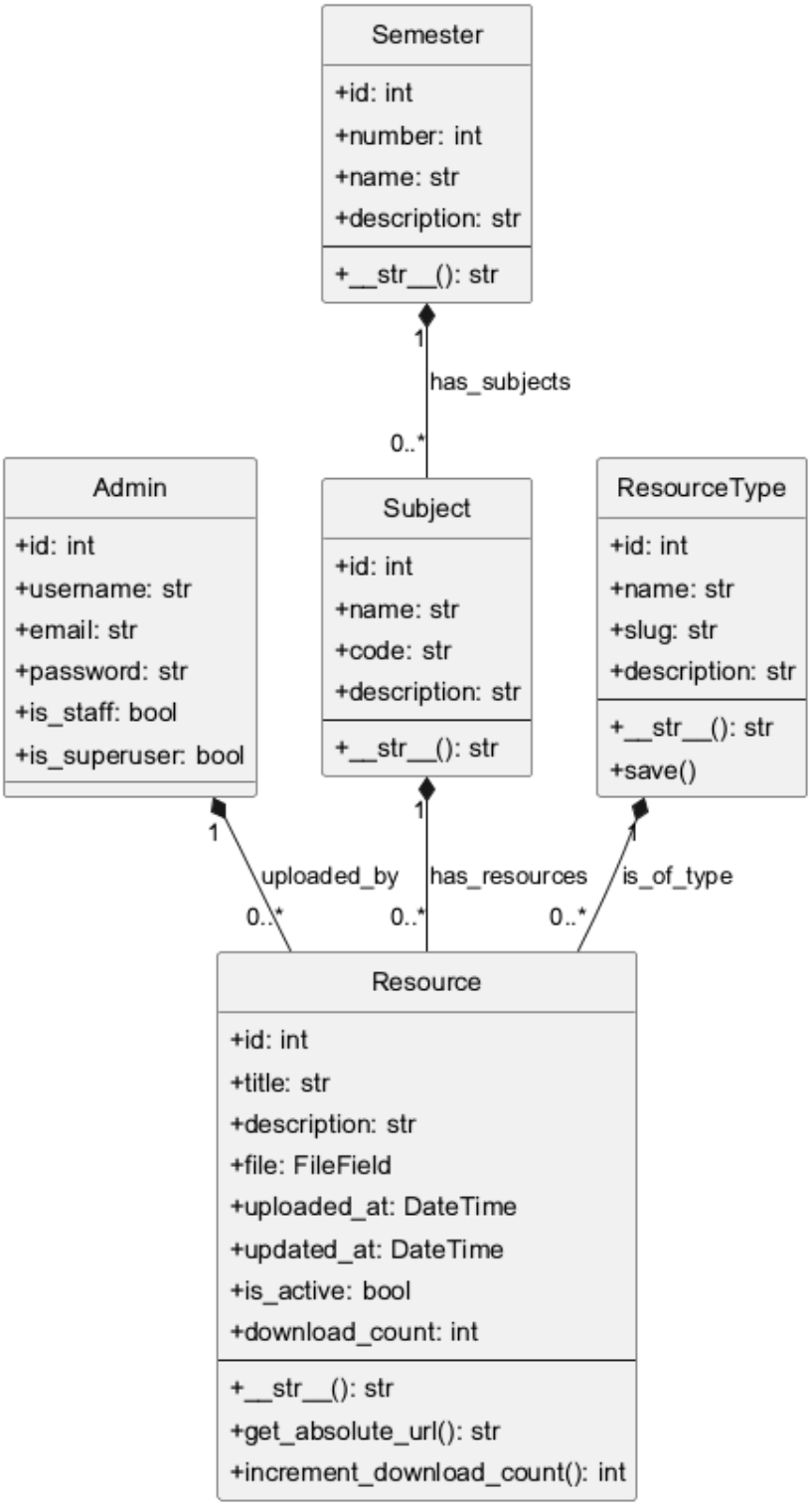


Figure 8 Refined Class Diagram

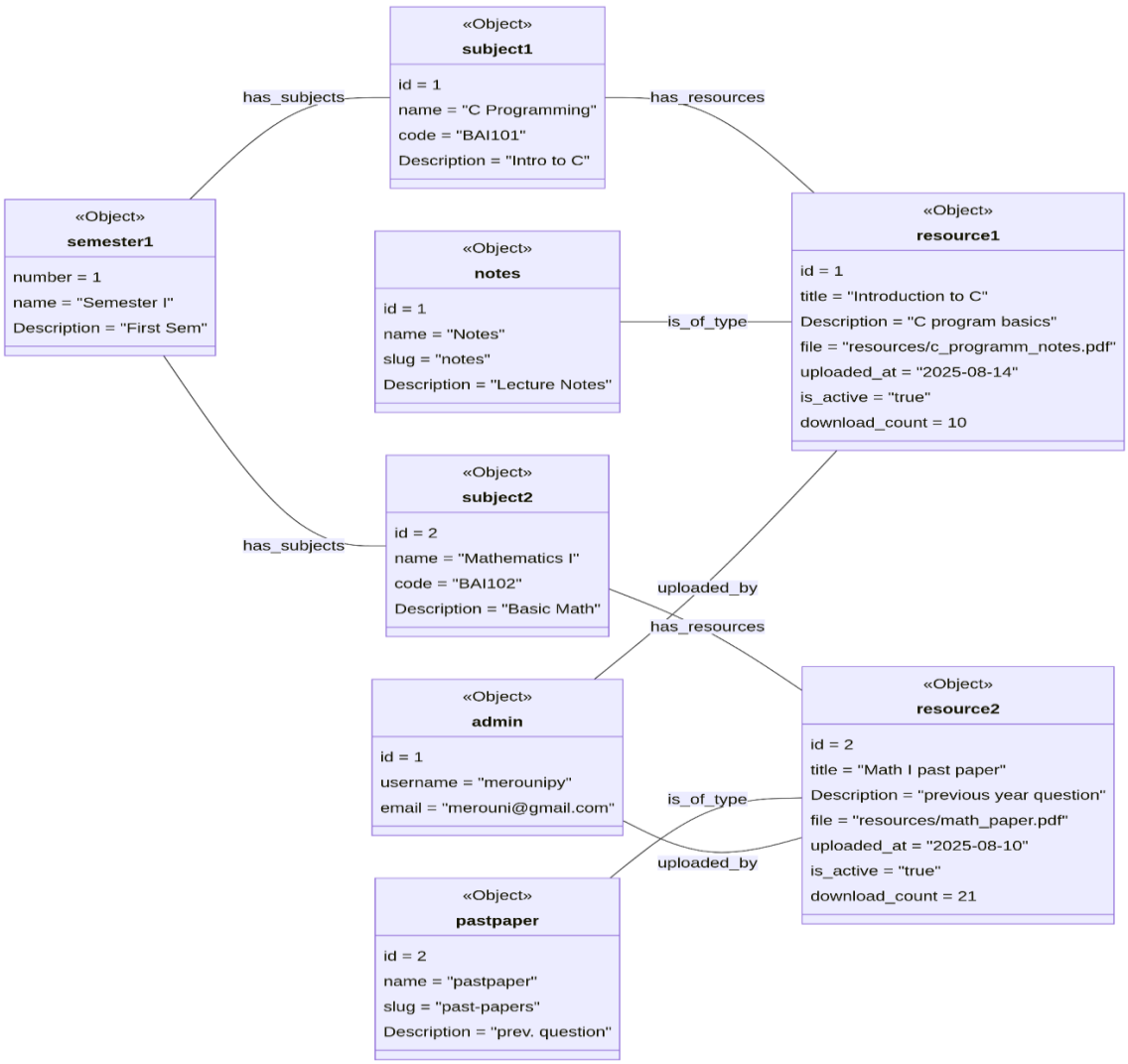


Figure 9 Refined Object Diagram

3.2.2 Component Diagram

The component diagram illustrates the structure of the Mero-uni system by showing its major components, their interfaces, and the relationships between them. It provides a high-level view of the software architecture, focusing on how different parts of the system are organized and interact

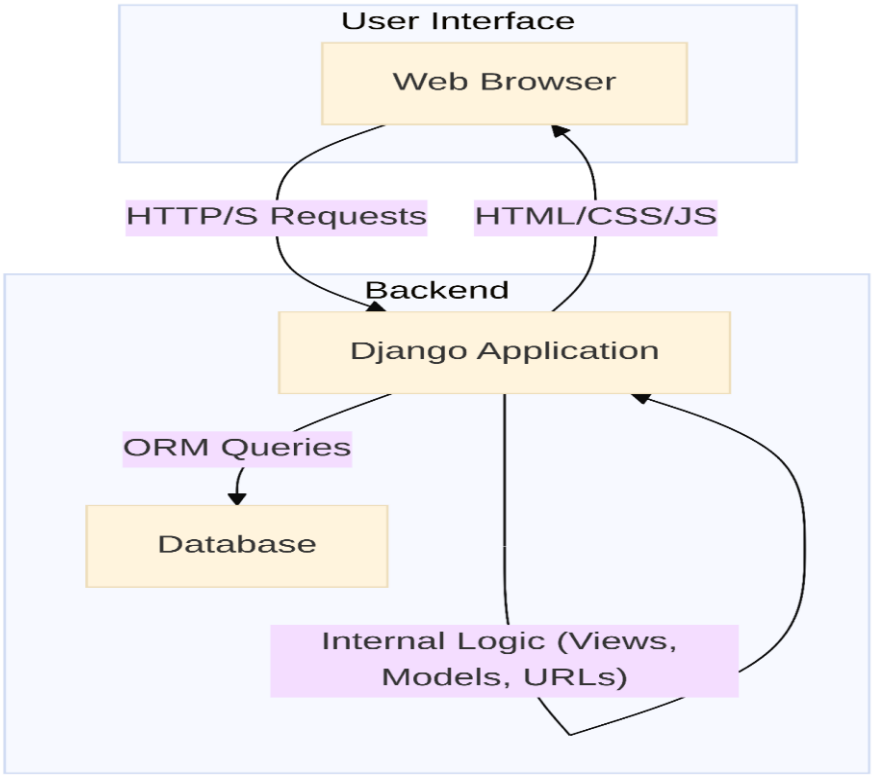


Figure 10 Component Diagram

Explanation:

Components:

Web Browser : Represents the client-side component, which is the user's interface to the Mero-uni application.

Django Application : Represents the core backend logic, including views, models, and URL routing, responsible for processing requests and interacting with the database.

Database : Represents the data storage component, where all application data (users, resources, semesters, subjects) is persistently stored.

Relationships: HTTP/S Requests : The Web Browser communicates with the Django Application using standard HTTP/S protocols for sending requests and receiving responses.

ORM Queries : The Django Application interacts with the Database primarily through Django's Object-Relational Mapper (ORM), which translates Python code into database queries.

Internal Logic : The Django Application has internal interactions between its various sub-components (views, models, URLs) to process requests and manage data.

HTML/CSS/JS : The Django Application sends rendered HTML, CSS, and JavaScript back to the Web Browser for display to the user.

3.2.3 Deployment Diagram

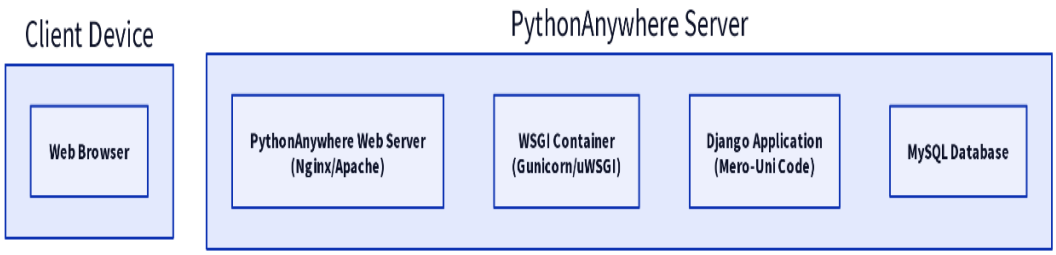


Figure 11 Deployement Diagram 1

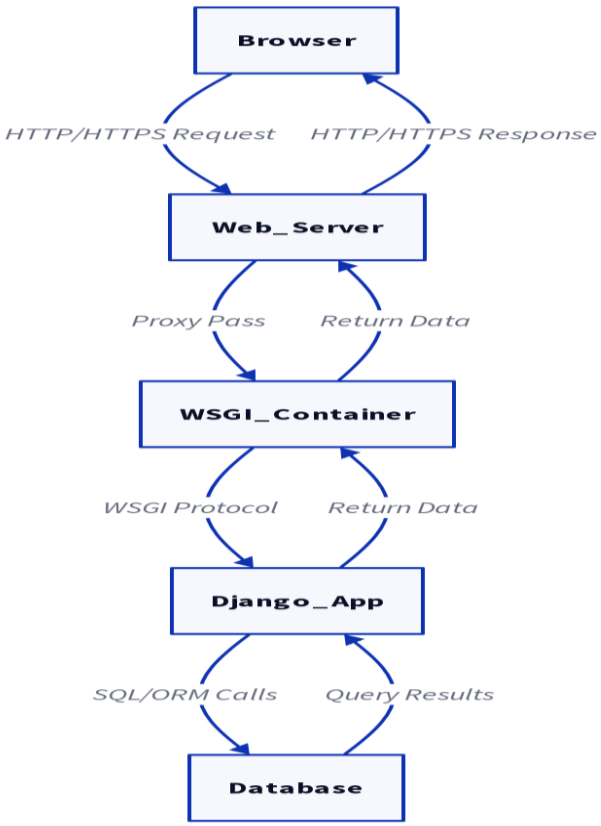


Figure 12Deployment Diagram 2

Chapter 4: Implementation and Testing

4.1. Implementation

The Mero-uni academic resource-sharing platform was developed using a combination of robust and widely adopted technologies, ensuring a scalable, maintainable, and secure application. The implementation phase involved translating the design specifications into functional code, focusing on modularity and adherence to best practices in web development.

4.1.1. Tools Used

The following tools, technologies, and frameworks were instrumental in the development of the Mero-uni project:

i. Programming Language: Python 3.13.5

Rationale: Python was chosen for its readability, extensive libraries, and strong community support, which accelerates development and simplifies complex tasks. Its versatility makes it suitable for web development, data processing, and system scripting.

ii. Web Framework: Django 5.2.4

Rationale: Django, a high-level Python web framework, was selected for its "batteries-included" philosophy, providing built-in features like an ORM, admin interface, and robust security mechanisms. Its MVT (Model-View Template) architecture promotes clean code separation and rapid development, aligning with the project's academic timeline and objectives.

iii. Database Management System: MySQL

iv. Frontend Technologies: HTML5, CSS3, JavaScript

Rationale: These standard web technologies provide the foundation for the user interface. HTML5 structures the content, CSS3 styles the presentation, and JavaScript enables interactive elements, ensuring a rich and responsive user experience across various web browsers.

v. Version Control System: Git

Rationale: Git was used for collaborative development, allowing the team to track changes, manage different versions of the codebase, and facilitate seamless integration of individual contributions. This ensured code integrity and efficient teamwork.

vi. Integrated Development Environment (IDE): Visual Studio Code,

Rationale: These IDEs provided comprehensive development environments with features like code highlighting, intelligent auto completion, debugging tools, and integrated terminal access, significantly enhancing developer productivity.

Other Tools & Platforms:

1. **Python Anywhere:** Hosting platform for deploying the Mero-uni project online.

4.1.2. Implementation Details of Modules

The Mero-uni application is structured into several interconnected modules, each responsible for specific functionalities. This modular design enhances maintainability, scalability, and allows for clear separation of concerns. The core modules and their implementation details are as follows:

Module 1: Resource Management Module

This module is central to the Mero-uni platform, enabling administrators to upload, categorize, and manage academic resources.

Functionalities: Resource upload (title, description, file, subject, resource type), resource editing, and resource deletion.

Implementation:

Models: The Resource model (defined in resources/models.py) stores metadata about each resource, including title , description , subject (ForeignKey to Subject), resource_type (ForeignKey to ResourceType), file (FileField for actual file storage), uploaded_by (ForeignKey to User), and download_count . Django's FileField handles file uploads and storage within the media/resources/ directory.

Forms: Django forms are used to create user-friendly interfaces for resource upload and editing, ensuring data validation and integrity.

Views: Dedicated views (e.g., upload_resource , edit_resource , delete_resource - implicitly handled by admin or future admin views) process form submissions, interact with the Resource model, and manage file operations.

Admin Interface: Django's powerful admin site (resources/admin.py) is extensively used by administrators for efficient management of Semester , Subject , ResourceType , and Resource instances, providing a ready-touse CRUD (Create, Read, Update, Delete) interface.

Module 2: **Browsing and Search Module**

This module facilitates the discovery and retrieval of resources by users.

Functionalities: Browsing resources by semester, subject, and resource type; searching resources by keywords; displaying resource details.

Implementation:

Views: Views like home , semester_detail , subject_detail , resources_by_type , and search_resources (defined in resources/views.py) handle user requests for browsing and searching. These views query the database using Django's ORM to retrieve relevant data.

URL Routing: Django's URL dispatcher (resources/urls.py) maps specific URL patterns to their corresponding views, enabling clean and logical navigation (e.g., /semester// , /subject// , /search/).

Templates: HTML templates (e.g., home.html , semester_detail.html , subject_detail.html , search_results.html) are used to render the retrieved data into user-friendly web pages. Django's template language allows for dynamic content generation.

Search Logic: The search_resources view utilizes Django's Q objects for complex database queries, allowing searches across multiple fields (title, description, subject name, subject code, resource type name) to provide comprehensive search results. d. Download Module This module manages the secure deli

Module 3: Download and Preview

This module manages the secure delivery of resource files to users, tracks download metrics and allows users to preview resources before downloading and securely download files.

Functionalities: Preview, Secure file download, incrementing download count.

Implementation: The download_resource view (resources/views.py) handles download requests. It retrieves the Resource object, increments its download_count using the increment_download_count method, and then serves the file using Django's HttpResponse with appropriate ContentDisposition headers. This ensures that files are downloaded as attachments with their original filenames.

4.2. Testing

Testing is an integral part of the software development process, ensuring that the Mero-uni application functions as intended, meets its requirements, and is free from critical defects. A multi-faceted testing strategy was employed, encompassing both unit and system testing.

4.2.1. Testing Strategy

The overall testing strategy for Mero-uni focused on verifying the correctness of individual components (unit testing) and ensuring that these components work together seamlessly as a complete system (system testing). The primary goal was to identify and rectify bugs early in the development cycle, thereby improving the overall quality and reliability of the application.

Table 2 Testing

Test Case ID	Module	Test Case Description	Pre-conditions	Test Steps	Expected Result	Test Type
TC-RM-001	Upload	Verify successful resource upload with valid data.	User is logged in as an Administrator. Valid PDF file (e.g., 5MB) is available.	1. Navigate to the Resource Upload page. 2. Fill in all required fields (Title, Description). 3. Select valid Semester, Subject, and Resource Type. 4. Upload the valid PDF file. 5. Click 'Submit'.	A success message is displayed. The resource appears in the browsing section with the correct metadata.	Functional

Test Case ID	Module	Test Case Description	Pre-conditions	Test Steps	Expected Result	Test Type
TC-RM-002	Upload	Verify upload failure with unsupported file type.	User is logged in as an Administrator. Invalid file type (e.g., .exe) is available.	1. Navigate to the Resource Upload page. 2. Attempt to upload the unsupported file type. 3. Click 'Submit'.	An error message ("Unsupported file type") is displayed. The file is not uploaded.	Functional
TC-RM-003	Upload	Verify upload failure with file exceeding size limit.	User is logged in as an Administrator. File exceeding the predefined size limit (e.g., 100MB) is available.	1. Navigate to the Resource Upload page. 2. Attempt to upload the oversized file. 3. Click 'Submit'.	An error message ("File size exceeds limit") is displayed. The file is not uploaded.	Functional
TC-RM-004	Categorization	Verify resource is correctly categorized and filterable.	TC-RM-001 is executed successfully.	1. Navigate to the Resource Browsing page. 2. Filter by the specific Semester, Subject, and Resource Type used in TC-RM-001.	The newly uploaded resource is visible in the filtered results.	Functional
Resource Access (Student/User)						

Test Case ID	Module	Test Case Description	Pre-conditions	Test Steps	Expected Result	Test Type
TC-RA-001	Browsing	Verify all resources are visible on the main browsing page.	Multiple resources exist across different categories.	Navigate to the main Resource Browsing page.	A list of all available resources is displayed, showing Title, Subject, Semester, and Resource Type.	Functional
TC-RA-002	Download	Verify successful resource download and count increment.	A resource exists with a known initial download_count.	1. Locate the resource. 2. Click the 'Download' button. 3. Check the file system for the downloaded file. 4. Refresh the resource page/list.	The file is successfully downloaded and opens correctly. The resource's download_count is incremented by 1.	Functional
TC-RA-003	Search	Verify search functionality by Title keyword.	Multiple resources exist with distinct titles.	1. Enter a full or partial keyword from a resource's title into the search bar. 2. Click 'Search'.	Only resources matching the keyword in the title are displayed.	Functional
TC-RA-004	Search	Verify search functionality by Description keyword.	Multiple resources exist with distinct descriptions.	1. Enter a full or partial keyword from a resource's description into the search bar. 2. Click 'Search'.	Only resources matching the keyword in the description are displayed.	Functional

Test Case ID	Module	Test Case Description	Pre-conditions	Test Steps	Expected Result	Test Type
TC-RA-005	Filtering	Verify filtering by Semester and Subject combination.	Resources are categorized by Semester and Subject.	1. Select a specific Semester and Subject from the filter dropdowns. 2. Apply the filter.	Only resources belonging to the selected Semester and Subject are displayed.	Functional
TC-NF-001	Responsiveness	Verify UI responsiveness on different screen sizes.	N/A	1. Access the application on a desktop browser. 2. Resize the browser window to simulate tablet and mobile views.	The layout adjusts correctly, all elements are accessible, and no horizontal scrolling is required.	Usability
TC-NF-002	Performance	Verify page load time for the main browsing page.	N/A	1. Clear browser cache. 2. Navigate to the main Resource Browsing page.	The page loads completely within an acceptable time limit (e.g., under 3 seconds).	Performance
TC-NF-003	Security	Verify input fields are protected against XSS attacks.	N/A	1. Attempt to enter a malicious script (e.g., <script>alert('XSS')</script>) into the Title or Description field during upload. 2. View the resource on the browsing page.	The script is sanitized or displayed as plain text, and no alert box or script execution occurs.	Security

Test Case ID	Module	Test Case Description	Pre-conditions	Test Steps	Expected Result	Test Type
TC-NF-004	Security	Verify file path is not exposed during download.	N/A	1. Initiate a resource download. 2. Monitor the network traffic or the download link.	The direct server file path (e.g., /home/ubuntu/files/notes.pdf) is not exposed to the client. The download is served through a secure, temporary, or obfuscated URL.	Security

Table 3 Unit Testing

Test ID	Method	Purpose	Test Steps	Expected Result	Outcome	Status
UT-002	Resource.is_admin_only()	Verify resource access control for non-admin users.	1. Create a resource marked as 'Admin Only'. 2. Call is_admin_only() method.	Returns True.	Passed	Passed
UT-003	Verify correct file extension extraction.	Verify correct file extension extraction.	1. Call get_file_extension() on a resource with file path 'notes/report.pdf'.	Returns '.pdf'.	Passed	Passed
UT-004	Resource.increment_download_count()	Verify download count increases correctly.	1. Get initial download count (e.g., 5). 2. Call increment_download_count(). 3. Get new count.	New count is initial count + 1 (e.g., 6).	Passed	Passed

4.2.4. Testing Tools

Django's Test Case and Client: Used for writing unit and integration tests, simulating HTTP requests to views and inspecting responses.

Manual Testing: Performed throughout the development cycle to verify UI/UX, responsiveness, and overall user experience.

Chapter 5: Conclusion and Future Recommendations

5.1. Lesson Learned and Outcome

The development of the Mero-uni academic resource-sharing platform has been an invaluable learning experience, offering profound insights into the complexities of software engineering, project management, and collaborative development. One of the foremost lessons learned was the critical importance of meticulous requirement analysis and clear communication within the development team. Early and continuous engagement with the problem domain, as articulated by the challenges faced by LTU students, proved essential in shaping a solution that genuinely addresses user needs. The iterative nature of the development process, while challenging, reinforced the value of agile principles in adapting to evolving requirements and unforeseen technical hurdles.

Technically, the project provided hands-on experience with the Django framework, deepening our understanding of its MVT architecture, ORM capabilities, and built-in security features. Navigating database interactions, managing file uploads, and implementing robust search functionalities presented practical challenges that honed our problem-solving skills. The process of integrating various components, from frontend user interfaces to backend logic database interactions and deployment using Python Anywhere, underscored the significance of modular design using UI/UX principles Learned the importance of a simple, intuitive interface for better user adoption and clean code practices for maintainability and scalability.

From a project management perspective, adhering to a structured timeline, as outlined in the BAI290 Project I Syllabus, was crucial. This necessitated effective task delegation, regular progress monitoring, and proactive identification of potential roadblocks. The collaborative aspect of working in a team highlighted the importance of version control systems (Git) for seamless code integration and conflict resolution, as well as the soft skills of effective communication and mutual support.

Outcome:

The Mero-uni project successfully delivered a functional web-based academic resource-sharing platform that effectively centralizes and organizes educational materials for the LTU community. The implemented system provides a user-friendly interface for browsing, searching, and downloading resources, while offering administrative tools for content management. It directly addresses the problem of fragmented resource sharing and lack of standardization, providing a more efficient and reliable alternative to informal methods. While the current iteration represents a foundational solution, it serves as a robust proof-of-concept demonstrating the feasibility and significant value of a dedicated academic resource platform.

5.2. Conclusion

The Mero-uni project set out to address challenges associated with academic resource sharing at Lumbini Technological University, characterized by disorganization, inconsistency, and limited accessibility. By developing a centralized, secure, and intuitive web-based platform, Mero-uni has successfully provided a comprehensive solution that streamlines the dissemination and retrieval of essential academic materials. The platform effectively addresses the core problems of fragmented resource availability and the absence of standardized repositories, thereby enhancing the overall learning experience for LTU students. Through precise system analysis, design, and implementation, Mero-uni offers key functionalities such as structured categorization by semester and subject, advanced search capabilities, and secure resource downloading. The architectural choices, leveraging Python, Django, and relational databases, have resulted in a robust and maintainable system. The successful development of Mero-uni underlines the potential of technology to significantly improve educational support systems and foster a more connected and efficient academic community. This project stands as a testament to the power of targeted technological solutions in overcoming real-world educational challenges.

5.3. Future Recommendations

While the current iteration of the Mero-uni platform provides a solid foundation, several avenues exist for future development and enhancement to further enrich its functionality, usability, and impact:

i. Develop mobile applications for iOS and Android to expand accessibility.
ii. Introduce collaborative tools, such as discussion forums.
iii. Implement a user feedback and rating system to ensure content quality.
iv. Integrate AI-driven recommendations for personalized resource suggestions.
v. Enhance security with multi-factor authentication

References

[1] A. Smith and B. Johnson, "Web-based academic resource sharing platforms: A comprehensive review," *IEEE Trans. Educ.*, vol. 65, no. 3, pp. 123–135, Aug. 2022.

[2] Bootstrap Team, *Bootstrap 5 Documentation*, 2025. [Online]. Available: https://getbootstrap.com/docs/5.0/

[3] C. Lee, D. Brown, and E. Wilson, "Collaborative learning environments in higher education: Design and implementation," *Computers & Education*, vol. 180, pp. 104–115, Mar. 2023.

[4] Django Software Foundation, *Django Documentation*, 2025. [Online]. Available: https://docs.djangoproject.com/

[5] F. Garcia and G. Martinez, "Django framework for educational web applications: Performance analysis," in *Proc. IEEE Int. Conf. Educational Technology*, San Francisco, CA, USA, 2023, pp. 45–52.

[6] HamroCSIT, *Academic Resource Platform for CSIT Students*, 2025. [Online]. Available: https://hamrocsit.com/

[7] H. Kim, I. Park, and J. Lee, "User experience design for academic resource platforms," in *Proc. ACM SIGCHI Conf. Hum. Factors Comput. Syst.*, New York, NY, USA, 2022, pp. 234–241.

[8] IEEE Std 12207-2017, *IEEE Standard for Software Engineering – Software Life Cycle Processes*, 2017.

[9] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2015.

[10] K. Thompson, "Design and implementation of collaborative academic platforms," Ph.D. dissertation, Dept. Computer. Sci., Stanford Univ., Stanford, CA, USA, 2022.

Appendix

Homepage/index page

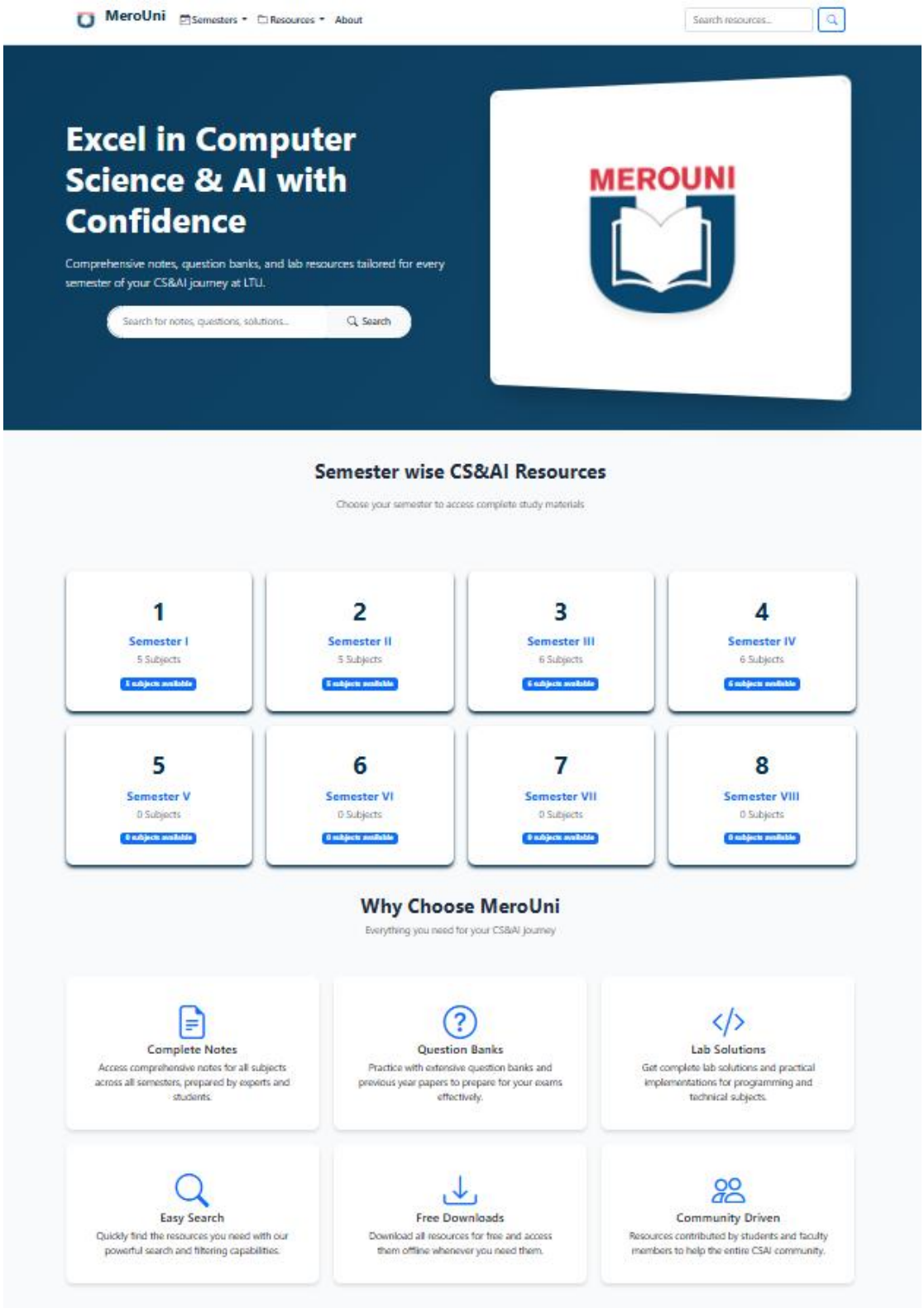


Figure 13 Homepage

Semester page

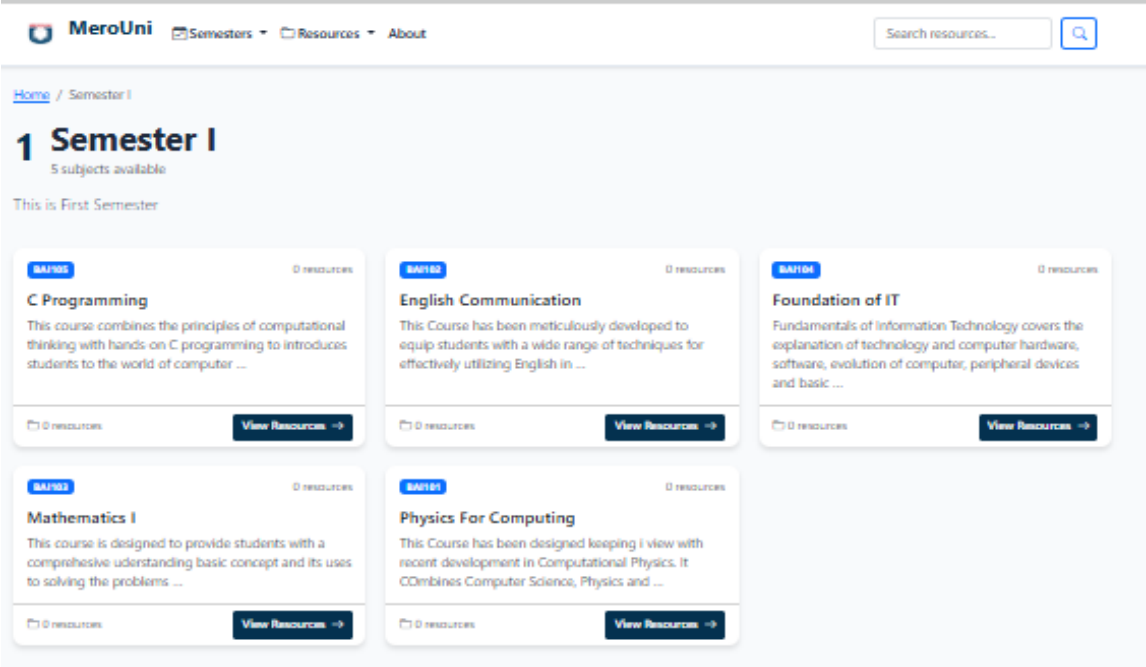


Figure 14 Semester Page

Subject Page

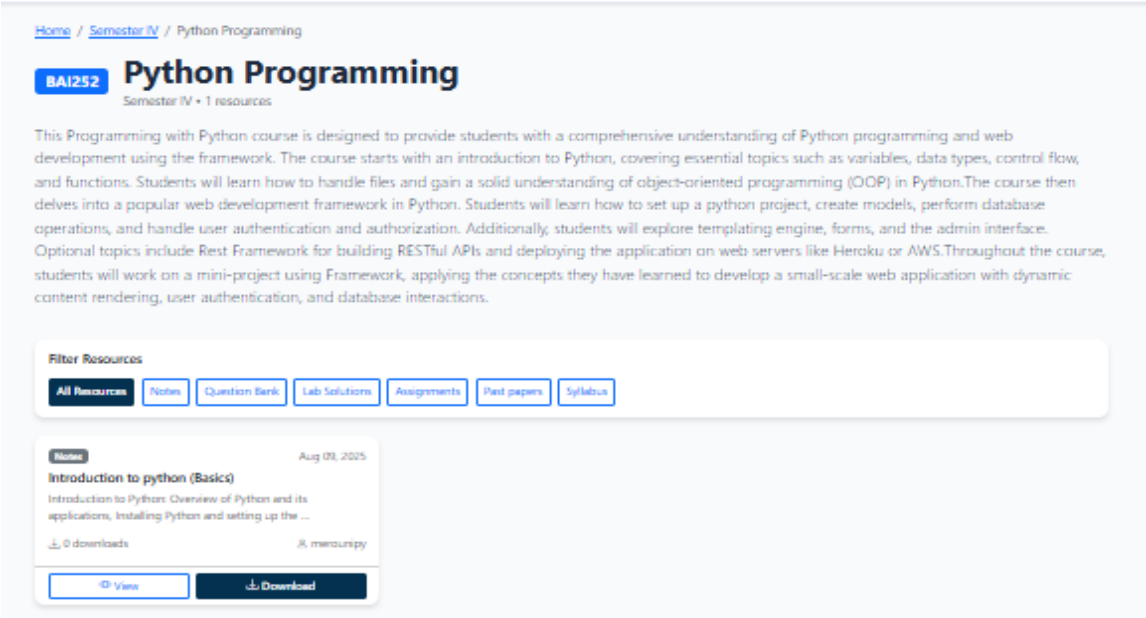


Figure 15 Subject Page

Resource Page

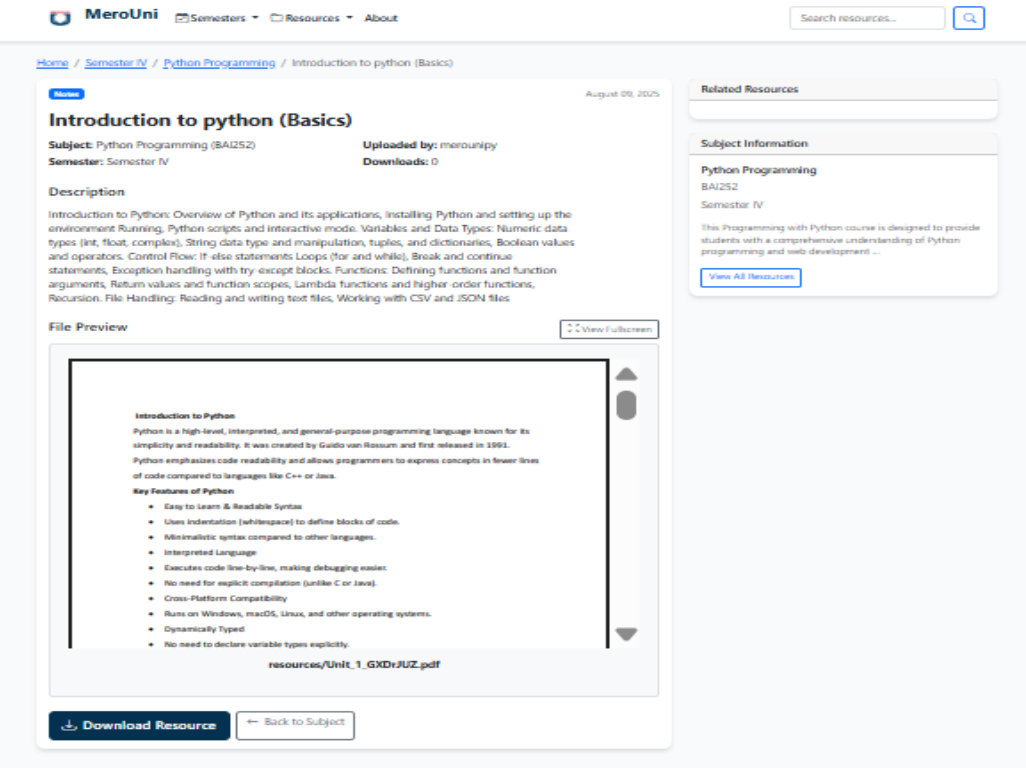


Figure 16 Resource Page

About Page

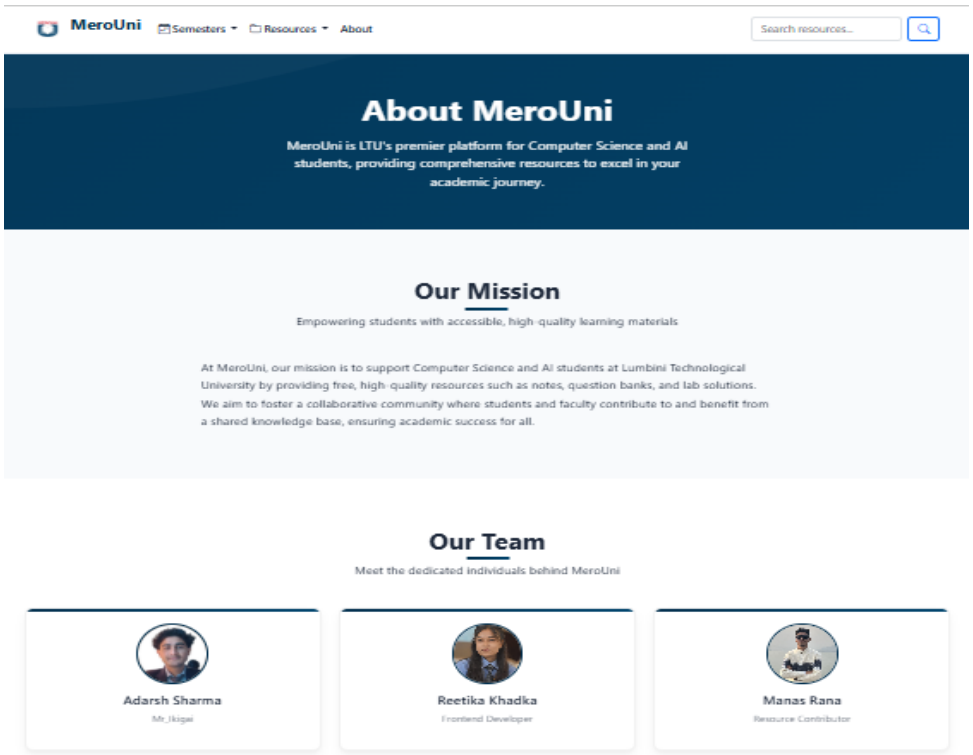


Figure 17 About Page

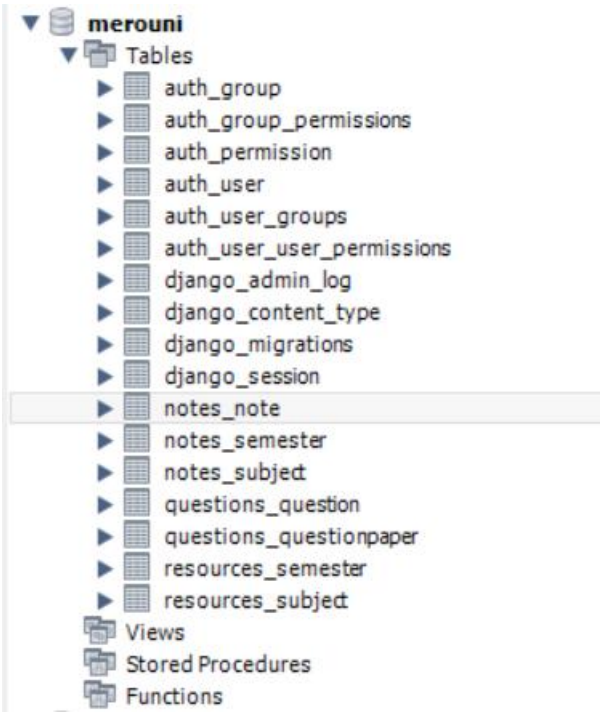


Figure 18 Database Mero-uni

Code Snippets:

Models.py

```
from django.db import models
from django.contrib.auth.models import User
from django.urls import reverse
from django.utils import timezone, translation

class Semester(models.Model):
    number = models.IntegerField(unique=True)
    name = models.CharField(max_length=50)
    description = models.TextField(blank=True)

    class Meta:
        ordering = ['number']

    def __str__(self):
        return f"{self.name}"

class Subject(models.Model):
    name = models.CharField(max_length=100)
    code = models.CharField(max_length=20, unique=True)
    semester = models.ForeignKey(Semester, on_delete=models.CASCADE,
related_name='subjects')
    description = models.TextField(blank=True)

    class Meta:
        ordering = ['name']

    def __str__(self):
        return f"{self.code} - {self.name}"

class ResourceType(models.Model):
    name = models.CharField(max_length=50, unique=True)
    slug = models.SlugField(max_length=50, unique=True)
    description = models.TextField(blank=True)

    def __str__(self):
        return self.name

    def save(self, *args, **kwargs):
        if not self.slug:
            from django.utils.text import slugify
            self.slug = slugify(self.name)
        super().save(*args, **kwargs)

class Resource(models.Model):
    title = models.CharField(max_length=200)
```

```
    description = models.TextField(blank=True)
    subject = models.ForeignKey(Subject, on_delete=models.CASCADE,
related_name='resources')
    resource_type = models.ForeignKey(ResourceType, on_delete=models.CASCADE,
related_name='resources')
    file = models.FileField(upload_to='resources/')
    uploaded_by = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='uploaded_resources')
    uploaded_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    is_active = models.BooleanField(default=True)
    download_count = models.PositiveIntegerField(default=0)

    class Meta:
        ordering = ['-uploaded_at']

    def __str__(self):
        return f"{self.title} - {self.subject.name}"

    def get_absolute_url(self):
        return reverse('resource_detail', kwargs={'pk': self.pk})

    def increment_download_count(self):
        self.download_count += 1
        self.save(update_fields=['download_count'])
```

Views.py

```
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.db.models import Q
from django.core.paginator import Paginator
from django.http import HttpResponse, Http404
from django.conf import settings
import os

from .models import Semester, Subject, Resource, ResourceType

def home(request):
    """Home page view"""
    semesters = Semester.objects.all().order_by('number')
    recent_resources = Resource.objects.filter(is_active=True).order_by('-
uploaded_at')[:6]
    resource_types = ResourceType.objects.all()
    context = {
```

```
        'semesters': semesters,
        'recent_resources': recent_resources,
        'resource_types': resource_types,
    }
    return render(request, 'resources/home.html', context)

def semester_detail(request, semester_number):
    """Semester detail page showing subjects"""
    semester = get_object_or_404(Semester, number=semester_number)
    subjects = semester.subjects.all().order_by('name')

    context = {
        'semester': semester,
        'subjects': subjects,
    }
    return render(request, 'resources/semester_detail.html', context)

def subject_detail(request, subject_id):
    """Subject detail page showing resources"""
    subject = get_object_or_404(Subject, id=subject_id)
    resources = Resource.objects.filter(subject=subject,
is_active=True).order_by('-uploaded_at')

    # Filter by resource type
    resource_type = request.GET.get('type')
    if resource_type:
        resources =
resources.filter(resource_type__name__icontains=resource_type)

    # Pagination
    paginator = Paginator(resources, 12)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    # Get all resource types for filtering
    resource_types = ResourceType.objects.all()

    context = {
        'subject': subject,
        'page_obj': page_obj,
        'resource_types': resource_types,
        'current_type': resource_type,
    }
    return render(request, 'resources/subject_detail.html', context)

def resource_detail(request, pk):
    """Resource detail page"""
    resource = get_object_or_404(Resource, pk=pk, is_active=True)
```

```
    context = {
        'resource': resource,
    }
    return render(request, 'resources/resource_detail.html', context)

def download_resource(request, pk):
    """Download resource file"""
    resource = get_object_or_404(Resource, pk=pk, is_active=True)

    # Increment download count
    resource.increment_download_count()

    # Get file path
    file_path = resource.file.path

    if os.path.exists(file_path):
        with open(file_path, 'rb') as fh:
            response = HttpResponse(fh.read(),
content_type="application/octet-stream")
            response['Content-Disposition'] = f'attachment;
filename="{os.path.basename(file_path)}"'
            return response
    else:
        raise Http404("File not found")

def search_resources(request):
    """Search resources"""
    query = request.GET.get('q', '')
    resources = Resource.objects.filter(is_active=True)

    if query:
        resources = resources.filter(
            Q(title__icontains=query) |
            Q(description__icontains=query) |
            Q(subject__name__icontains=query) |
            Q(subject__code__icontains=query) |
            Q(resource_type__name__icontains=query)
        ).distinct()

    # Pagination
    paginator = Paginator(resources, 12)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    context = {
        'page_obj': page_obj,
        'query': query,
```

```
        'total_results': resources.count(),
    }
    return render(request, 'resources/search_results.html', context)

def resources_by_type(request, type_slug):
    """Show resources by type"""
    type_mapping = {
        'notes': 'Notes',
        'question-bank': 'Question Bank',
        'lab-solutions': 'Lab Solutions',
        'assignments': 'Assignments',
        'past-papers': 'Past papers',
        'syllabus': 'Syllabus',
    }

    type_name = type_mapping.get(type_slug)
    if not type_name:
        raise Http404("Resource type not found")

    resource_type = get_object_or_404(ResourceType, name=type_name)
    resources = Resource.objects.filter(resource_type=resource_type,
is_active=True).order_by('-uploaded_at')

    # Filter by semester if specified
    semester = request.GET.get('semester')
    if semester:
        resources = resources.filter(subject__semester__number=semester)

    # Pagination
    paginator = Paginator(resources, 12)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    # Get all semesters for filtering
    semesters = Semester.objects.all().order_by('number')

    context = {
        'resource_type': resource_type,
        'page_obj': page_obj,
        'semesters': semesters,
        'current_semester': semester,
    }
    return render(request, 'resources/resources_by_type.html', context)

def about(request):
    return render(request, 'resources/about.html')
```

Home.html

```
{% extends 'base/base.html' %}
{% load static %}
{% block title %}Home - MeroUni Resources{% endblock %}
{% block content %}
<!-- Hero Section -->
<section class="hero-section">
    <div class="container">
        <div class="row align-items-center hero-content">
            <div class="col-lg-6 hero-text">
                <h1 class="display-4 fw-bold mb-4">Excel in Computer Science &
AI with Confidence</h1>
                <p class="lead mb-4">Comprehensive notes, question banks, and
lab resources tailored for every semester of your CS&AI journey at LTU.</p>

                <!-- Search Form -->
                <form class="search-form" method="GET" action="{% url
'search_resources' %}">
                    <div class="input-group">
                        <input type="text" class="form-control search-input"
name="q" placeholder="Search for notes, questions, solutions..." aria-
label="Search">
                        <button class="btn btn-light search-btn"type="submit">
                            <i class="bi bi-search me-2"></i>Search
                        </button>
                    </div>
                </form>
            </div>
            <div class="col-lg-6">
                <div class="hero-image-container text-center">
                    <img src="{% static 'Merouni-Logo.png' %}" alt="Students
learning computer science" class="img-fluid hero-image" loading="lazy"
width="600" height="400">
                </div>
            </div>
        </div>
    </div>
</section>

<!-- Semester Cards -->
<section class="container">
    <div class="row mb-5">
        <div class="col-12 text-center">
            <h2 class="fw-bold mb-4">Semester wise CS&AI Resources</h2>
            <p class="text-muted mb-5">Choose your semester to access complete
study materials</p>
        </div>
```

```
    </div>

    <div class="row g-4">
        {% for semester in semesters %}
        <div class="col-lg-3 col-md-4 col-sm-6">
            <a href="{% url 'semester_detail' semester.number %}" class="text-
decoration-none">
                <div class="semester-card">
                    <div class="semester-number">{{ semester.number }}</div>
                    <h5 class="fw-bold mb-2">{{ semester.name }}</h5>
                    <p class="text-muted mb-3">{{ semester.subjects.count }}
Subjects</p>
                    <div class="d-flex justify-content-center">
                        <span class="badge bg-primary">{{
semester.subjects.count }} subjects available</span>
                    </div>
                </div>
            </a>
        </div>
        {% endfor %}
    </div>
</section>

<!-- Features Section -->
<section class="container mt-5">
    <div class="row">
        <div class="col-12 text-center mb-5">
            <h2 class="fw-bold">Why Choose MeroUni</h2>
            <p class="text-muted">Everything you need for your CS&AI
journey</p>
        </div>
    </div>
    <div class="row g-4">
        <div class="col-lg-4 col-md-6">
            <div class="card h-100 text-center p-4">
                <div class="card-body">
                    <i class="bi bi-file-earmark-text display-4 text-primary
mb-3"></i>
                    <h5 class="card-title">Complete Notes</h5>
                    <p class="card-text">Access comprehensive notes for all
subjects across all semesters, prepared by experts and students.</p>
                </div>
            </div>
        </div>
        <div class="col-lg-4 col-md-6">
            <div class="card h-100 text-center p-4">
                <div class="card-body">
```

```
                    <i class="bi bi-question-circle display-4 text-primary mb-
3"></i>
                    <h5 class="card-title">Question Banks</h5>
                    <p class="card-text">Practice with extensive question
banks and previous year papers to prepare for your exams effectively.</p>
                </div>
            </div>
        </div>
        <div class="col-lg-4 col-md-6">
            <div class="card h-100 text-center p-4">
                <div class="card-body">
                    <i class="bi bi-code-slash display-4 text-primary mb-
3"></i>
                    <h5 class="card-title">Lab Solutions</h5>
                    <p class="card-text">Get complete lab solutions and
practical implementations for programming and technical subjects.</p>
                </div>
            </div>
        </div>
        <div class="col-lg-4 col-md-6">
            <div class="card h-100 text-center p-4">
                <div class="card-body">
                    <i class="bi bi-search display-4 text-primary mb-3"></i>
                    <h5 class="card-title">Easy Search</h5>
                    <p class="card-text">Quickly find the resources you need
with our powerful search and filtering capabilities.</p>
                </div>
            </div>
        </div>
       <div class="col-lg-4 col-md-6">
            <div class="card h-100 text-center p-4">
                <div class="card-body">
                    <i class="bi bi-download display-4 text-primary mb-3"></i>
                    <h5 class="card-title">Free Downloads</h5>
                    <p class="card-text">Download all resources for free and
access them offline whenever you need them.</p>
                </div>
            </div>
        </div>
        <div class="col-lg-4 col-md-6">
            <div class="card h-100 text-center p-4">
                <div class="card-body">
                    <i class="bi bi-people display-4 text-primary mb-3"></i>
                    <h5 class="card-title">Community Driven</h5>
                    <p class="card-text">Resources contributed by students and
faculty members to help the entire CSAI community.</p>
                </div>
            </div>
```

```
        </div>
    </div>
</section>

<!-- Recent Resources -->
{% if recent_resources %}
<section class="container mt-5">
    <div class="row">
        <div class="col-12">
            <h3 class="fw-bold mb-4">Recently Added Resources</h3>
        </div>
    </div>
    <div class="row g-4">
        {% for resource in recent_resources %}
        <div class="col-lg-4 col-md-6">
            <div class="card h-100">
                <div class="card-body">
                    <div class="d-flex justify-content-between align-items-
start mb-2">
                        <span class="badge bg-secondary">{{
resource.resource_type.name }}</span>
                        <small class="text-muted">{{
resource.uploaded_at|date:"M d" }}</small>
                    </div>
                    <h6 class="card-title">{{ resource.title }}</h6>
                    <p class="card-text text-muted small">{{
resource.subject.name }}</p>
                    <p class="card-text small">{{
resource.description|truncatewords:15 }}</p>
                </div>
                <div class="card-footer bg-transparent">
                    <div class="d-flex justify-content-between align-items-
center">
                        <small class="text-muted">
                            <i class="bi bi-download me-1"></i>{{
resource.download_count }} downloads
                        </small>
                        <a href="{% url 'resource_detail' resource.pk %}"
class="btn btn-sm btn-outline-primary">View</a>
                    </div>
                </div>
            </div>
        </div>
        {% endfor %}
    </div>
</section>
{% endif %}
{% endblock %}
```

Supervision Log Record

Lumbini Technological University
Institute of Engineering and Information Technology
Banke, Lumbini Province

Project Supervision Log Record

Program: B.Tech in CS&AI **Year/Semester:** II/IV **Subject:** BAI290: Project I

Project Title: Mproject : Alpha Storing platform

Supervisor Name: Prabhat Chand

SN	Meeting Agenda	Action	Suggestion	Date of Meeting	Signature of Supervisor
1	Proposal	Proposal report check	Make proposal format presentable	2082/03/10	
2	Proposal report review	Proposal report reviewed	Proposal is accepted.	2082/03/13	
3.	UI design	UI design review	Add storing search option	2082/03/20	
4.	Database design	Reviewed database design	Create database	2082/03/30	
			to handle file		
5.	Review of database interaction with UI	Database interaction checked	Make a dynamic form to upload files and download button to download files.	2082/04/09	

Figure 19 Log Records 1

Lumbini Technological University
Institute of Engineering and Information Technology
Banke, Lumbini Province

Project Supervision Log Record

Program: B.Tech in CS&AI **Year/Semester:** II/IV **Subject:** BAI290: Project I

Project Title: MeroUni: notes sharing platform

Supervisor Name: Prabhik Chand

SN	Meeting Agenda	Action	Suggestion	Date of Meeting	Signature of Supervisor
6.	Review of whole system	Reviewed all the feature of the system.	Everything is fine.	2032/07/13	Prabhik
			make necessary change		
			as better design.		

Figure 20 Log Record 2